

OPERATING SYSTEMS

Complete Placement Preparation Guide

Premium Technical Reference for Software Engineering Interviews

Covers: Process Management · CPU Scheduling · Synchronization · Deadlocks

Memory Management · Virtual Memory · File Systems · I/O · Linux Internals

50 Top Interview Q&A; · 25 Must-Practice Problems · Worked Examples

Read 2 days before your placement interview for maximum impact.

TABLE OF CONTENTS

Chapter 1	Introduction to Operating Systems	3
Chapter 2	Process Management — Fundamentals	5
Chapter 3	Threads & Multithreading	8
Chapter 4	CPU Scheduling	10
Chapter 5	Process Synchronization	14
Chapter 6	Deadlocks	18
Chapter 7	Memory Management — Basics	21
Chapter 8	Paging & Segmentation	23
Chapter 9	Virtual Memory	25
Chapter 10	File Systems	28
Chapter 11	I/O Systems & Disk Management	31
Chapter 12	Linux & System Programming Basics	33
Chapter 13	Advanced OS Concepts	34
Chapter 14	Top 50 Interview Questions — Detailed Answers	35
Chapter 15	25 Must-Practice Problems	48

CHAPTER 1: Introduction to Operating Systems

1.1 What is an Operating System?

An **Operating System (OS)** is a large, complex software program that manages computer hardware and software resources, and provides common services for computer programs. Think of it as the master resource manager: every program you run — a browser, a Python script, a game — depends entirely on the OS to interact with the underlying hardware (CPU, RAM, disk, GPU, network). Without an OS, every developer would have to write device drivers, manage memory pages, and handle interrupts themselves — an intractable problem at scale.

The OS sits between user applications (user space) and the raw hardware (kernel space). Its three primary **goals** are: **Convenience** — making the computer easy to use through abstractions like files and processes; **Efficiency** — utilizing hardware resources optimally (maximizing CPU throughput, minimizing wasted memory); and **Evolution** — allowing new features to be added and hardware to change without rewriting applications.

1.1.1 Core OS Functions

Function	Description
Resource Management	Allocates CPU time, memory, I/O devices, and files among competing processes.
Process Management	Creates, schedules, suspends, resumes, and terminates processes and threads.
Memory Management	Tracks which memory is in use, allocates/deallocates memory, handles virtual memory.
File Management	Creates, deletes, opens, closes files; manages directories and access permissions.
I/O Management	Buffers, caches, spools I/O operations; provides uniform interface to diverse devices.
Security & Protection	Controls access to resources; authenticates users; enforces isolation between processes.
User Interface	Provides CLI (bash, zsh) or GUI (Windows Explorer, GNOME) for human interaction.

1.2 Types of Operating Systems

OSes are classified by their design goals and deployment environment. Understanding these types is essential because interviewers often ask about trade-offs between latency, throughput, and resource utilisation.

Type	Characteristics
Batch OS	No user interaction; jobs collected, processed in batches. Early IBM systems. High throughput, poor responsiveness.
Multiprogrammed	Multiple jobs loaded in memory simultaneously; CPU switches to another when one does I/O. Improves CPU utilisation.
Multitasking / Time-Sharing	Extension of multiprogramming with rapid CPU switching (time quantum); users get illusion of simultaneous execution. UNIX.
Multiprocessing	Multiple CPUs share memory (SMP). Parallel execution. Higher throughput and fault tolerance.

Type	Characteristics
Distributed OS	Multiple networked computers coordinated as a single coherent system (e.g., Google's Borg / Kubernetes).
Real-Time OS (Hard)	Must meet strict time deadlines — missing a deadline is a catastrophic failure (pacemakers, aircraft autopilot).
Real-Time OS (Soft)	Deadlines are important but not catastrophic to miss (video streaming, online games).
Embedded OS	Designed for embedded devices with limited resources: ROM-based, minimal footprint (VxWorks, FreeRTOS, Zephyr).
Mobile OS	Optimised for battery life, touch input, cellular radios (Android/Linux kernel, iOS/XNU kernel).

1.3 Kernel Architecture

The **kernel** is the core of the OS — the only software running at the highest privilege level (Ring 0 on x86). It directly controls hardware. Different kernel designs make different trade-offs between performance and modularity.

Kernel Type	Description, Trade-offs, Examples
Monolithic	Entire OS runs as a single program in kernel space. All services (file system, drivers, scheduler) are in one large binary. Fast (no context switches between components) but hard to maintain . Linux, BSD.
Microkernel	Only minimal services (IPC, basic scheduling, memory management) in kernel. All else (file system, drivers) runs in user space as servers. Modular, fault-tolerant but slower (many IPC calls). MINIX, QNX, GNU Hurd.
Hybrid	Monolithic base + some services moved out. Balance between performance and modularity. Windows NT, macOS (XNU = Mach microkernel + BSD subsystem).
Layered	OS organized as hierarchy of layers; each layer uses services of the layer below. Easy to debug but poor performance. THE system (Dijkstra, 1968).
Exokernel	Extremely minimal kernel that only multiplexes hardware securely. Applications control their own resource management. Research systems (MIT Exokernel). Maximum flexibility.

1.4 User Mode vs. Kernel Mode

Modern CPUs have at least two privilege levels to protect the OS from buggy or malicious programs. In **User Mode** (Ring 3), software cannot execute privileged instructions — it cannot directly read/write I/O ports, modify interrupt tables, or access arbitrary physical memory addresses. In **Kernel Mode** (Ring 0), the OS can execute any CPU instruction and access any memory address.

When a user program needs an OS service (e.g., read a file), it triggers a **mode switch** via a system call. The CPU saves the current register state, raises privilege level to Ring 0, jumps to a predefined OS handler, executes the service, lowers privilege back to Ring 3, and returns. This boundary is fundamental to OS security — it ensures user programs cannot corrupt the kernel.

1.5 System Calls

A **system call** is the mechanism by which a user-space program requests a service from the kernel. System calls form the API boundary between user applications and the OS. On x86-64 Linux, a system call is triggered by the syscall instruction; on x86, by int 0x80.

Category	Common Linux System Calls
Process Control	fork(), exec(), exit(), wait(), kill(), getpid()
File Manipulation	open(), close(), read(), write(), lseek(), stat(), chmod()
Device Manipulation	ioctl(), read(), write() on device files
Information Maintenance	getpid(), alarm(), sleep(), time(), gettimeofday()
Communication (IPC)	pipe(), socket(), shmget(), mmap(), msgget(), semget()
Protection	chmod(), chown(), umask(), setuid(), getuid()

Parameter passing methods: (1) **Registers** — fastest; parameters in CPU registers (Linux uses rdi, rsi, rdx, r10, r8, r9 for first 6 args); (2) **Memory block** — parameters stored in a memory block and address passed in register; (3) **Stack** — parameters pushed on program stack, kernel pops them.

1.6 Booting Process

When you power on a computer: (1) **Power-On Self Test (POST)** — firmware (BIOS/UEFI) runs hardware diagnostics; (2) **Boot device selection** — firmware reads the boot device's Master Boot Record (MBR) or GPT; (3) **Bootloader** (GRUB2 on Linux) is loaded — a small program that loads the OS kernel into RAM; (4) **Kernel initialization** — decompresses itself, sets up memory (page tables, kernel space), initializes device drivers, mounts root filesystem; (5) **Init / systemd** — PID 1 process starts, launches all user-space services.

QUICK REVISION BOX

- ✓ OS manages CPU, memory, I/O, files, security — it's the hardware abstraction layer.
- ✓ Three OS goals: Convenience, Efficiency, Evolution.
- ✓ Monolithic kernels (Linux) are fast; Microkernels (QNX) are modular and fault-tolerant; Hybrid (Windows, macOS) balance both.
- ✓ User Mode vs Kernel Mode: mode switch happens on every system call — this boundary is security-critical.
- ✓ System calls are the only legal way for user programs to request kernel services.
- ✓ Six system call categories: Process, File, Device, Info, Communication, Protection.
- ✓ Boot sequence: POST → Bootloader → Kernel init → systemd/init.

CHAPTER 2: Process Management — Fundamentals

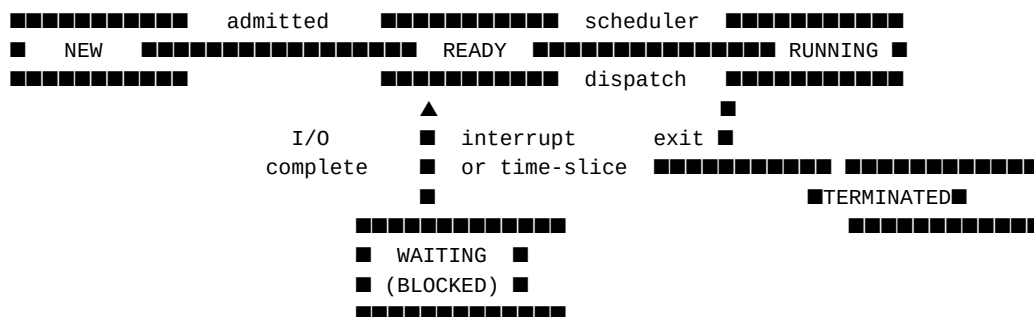
2.1 Program vs. Process

A **program** is a *passive*, static entity — just a file on disk containing compiled machine instructions (e.g., /usr/bin/python3). A **process** is an *active*, dynamic entity — a program in execution with its own private address space, register values, open file descriptors, and program counter. The same program can spawn multiple processes simultaneously (e.g., two terminal windows each running bash).

Aspect	Program	Process
Nature	Passive (static file on disk)	Active (executing instance)
Storage	Secondary storage (HDD/SSD)	Primary memory (RAM)
Lifetime	Permanent until deleted	Temporary — lives while running
Address Space	None (just bytes in a file)	Code, Data, Heap, Stack segments
Multiplicity	One copy on disk	Many processes from same program

2.2 Process States & State Transition Diagram

A process passes through well-defined states during its lifetime. Understanding state transitions is crucial — interviewers often ask "in which state does a process spend the most time" (usually **Ready** or **Waiting**) or "what triggers each transition."



Extended States (Swapping):

READY ■■swap out■■■ SUSPENDED READY (process in swap space, ready to run)

WAITING ■■swap out■■■ SUSPENDED WAITING (blocked + swapped out to disk)

State	Description
New	Process is being created — PCB allocated, resources being assigned.
Ready	Process is in memory, waiting for CPU. Has all resources except CPU.
Running	Process instructions are being executed on the CPU. Only one per core.
Waiting / Blocked	Process is waiting for an event (I/O completion, semaphore, signal).
Terminated	Process has finished execution. PCB still exists until parent calls wait().
Suspended Ready	Process swapped out to disk but was ready. Will resume when swapped back.
Suspended Waiting	Process swapped out while blocked. Must wait for both event and swap-in.


```

pid_t pid = fork();           // Creates child process (copy of parent)

if (pid < 0) {
    // Fork failed
    perror("fork");
    return 1;
} else if (pid == 0) {
    // Child process: pid == 0
    execl("/bin/ls", "ls", "-la", NULL); // Replace with new program
    // exec never returns if successful
} else {
    // Parent process: pid = child's PID
    int status;
    waitpid(pid, &status, 0); // Wait for child to finish
    // Reap child → prevents zombie
}
return 0;
}

```

fork() creates an exact copy of the calling process. The child gets a copy of the parent's address space (using Copy-on-Write optimisation). Both parent and child continue from the instruction after `fork()`. **exec()** replaces the current process image with a new program — it does not create a new process. The combination `fork()+exec()` is how shells launch programs.

2.7 Zombie and Orphan Processes

A **zombie process** is a process that has finished execution (called `exit()`) but its PCB still exists in the OS because the parent has not yet called `wait()` to read its exit status. Zombies consume no CPU or memory resources — only a PID and a PCB entry. They are dangerous if they accumulate (PID table fills up). **Prevention:** parent calls `wait()` or `waitpid()`; or installs a `SIGCHLD` signal handler to reap children automatically.

An **orphan process** is a process whose parent has died before the child. Linux re-parents orphans to **init (PID 1)** / **systemd**, which periodically calls `wait()` for all its adopted children. Orphans are generally not problematic; zombies are.

2.8 Inter-Process Communication (IPC)

Processes have isolated address spaces by design (security, fault isolation). When processes need to cooperate or exchange data, they use **IPC mechanisms**.

Shared Memory

Shared Memory is the fastest IPC mechanism. A region of memory is mapped into the address spaces of two or more processes. One process writes to the shared region; another reads it. The OS only sets up the mapping — after that, all data transfer happens at memory speed (no kernel involvement per message). **Disadvantage:** shared memory requires explicit synchronisation (semaphores, mutexes) to prevent race conditions.

Message Passing

Message Passing lets processes communicate by sending and receiving messages. The OS manages the channel. **Direct communication:** processes name each other explicitly (`send(P, message)`, `receive(Q, message)`). **Indirect communication (mailbox):** messages sent to a named mailbox (`send(A, message)`) — multiple processes can share a mailbox. **Synchronous (blocking):** sender blocks until receiver gets message. **Asynchronous (non-blocking):** sender continues immediately after sending.

Pipes

```

/* Ordinary (unnamed) pipe – only between related processes */
int fd[2];
pipe(fd);           // fd[0]=read end, fd[1]=write end
if (fork() == 0) {

```



```

        close(fd[1]);                // child reads
        read(fd[0], buf, sizeof(buf));
    } else {
        close(fd[0]);                // parent writes
        write(fd[1], "hello", 5);
    }

/* Named pipe (FIFO) – between unrelated processes */
mkfifo("/tmp/myfifo", 0666);        // create named pipe
// Process A: open("/tmp/myfifo", O_WRONLY);
// Process B: open("/tmp/myfifo", O_RDONLY);

```

IPC Mechanism	Characteristics & Use Cases
Shared Memory	Fastest; zero-copy; needs explicit sync. Best for large data.
Pipes (Unnamed)	Unidirectional; only between parent/child; kernel-buffered (64KB default).
Named Pipes (FIFO)	Unidirectional; any two processes; accessed as filesystem path.
Message Queues	Kernel-maintained queue; messages have types; persistent after sender exits.
Sockets	Full-duplex; works across network; TCP/UDP; most flexible but highest overhead.
Signals	Asynchronous notification; integer signal number; no data payload.

QUICK REVISION BOX

- ✓ Program is passive (disk); Process is active (RAM) with PCB, state, and address space.
- ✓ 5 core states: New → Ready → Running → Waiting → Terminated; plus Suspended variants.
- ✓ PCB stores everything about a process: PID, PC, registers, memory info, open files.
- ✓ Context switch saves/restores PCB — pure overhead; mode switch (user↔kernel) is cheaper.
- ✓ Process memory: Text (code) → Data → BSS → Heap (↑) ... Stack (↓).
- ✓ fork() copies process; exec() replaces process image; wait() reaps zombie children.
- ✓ Zombie = exited but not waited on; Orphan = parent died → re-parented to init.
- ✓ Shared memory is fastest IPC; sockets are most flexible; pipes are simplest.

CHAPTER 3: Threads & Multithreading

3.1 What is a Thread?

A **thread** is the basic unit of CPU utilisation — the smallest sequence of instructions that a CPU scheduler can independently schedule. A single process can contain multiple threads, all sharing the same address space (code, data, heap, open files) but each having its own **program counter, registers, and stack**. Threads are sometimes called **lightweight processes (LWP)** because they are cheaper to create, schedule, and communicate than full processes.

■ INTERVIEW TIP: Thread vs Process

A process is a resource container (address space, files, PID). A thread is an execution unit within a process. Multiple threads share one process's resources. Creating a thread is ~10× faster than creating a process. Context switching between threads of the same process is also faster — no address space switch.

3.2 Process vs Thread — Detailed Comparison

Aspect	Process	Thread
Creation overhead	High (fork copies page tables, file descriptors)	Low (only stack + registers allocated)
Memory sharing	Private address space per process	Shared address space; only stack is private
Context switch cost	High (TLB flush, page table switch)	Low (same page tables, no TLB flush)
Communication	Requires IPC (pipe, shared mem, socket)	Direct — shared memory within process
Crash isolation	Crash in one process doesn't affect others	Crash in one thread can kill entire process
Synchronisation	OS-level mechanisms (shared memory+semaphore)	Need mutex/semaphore for shared data
Use case	Isolated tasks, security boundaries	Concurrent tasks sharing data (web server workers)

3.3 Benefits of Multithreading

- Responsiveness — UI thread stays interactive while worker thread runs long computation.
- Resource sharing — threads share code and data segments; no IPC overhead for data exchange.
- Economy — thread creation ~10× faster than process creation; cheaper context switches.
- Scalability — on multi-core CPUs, threads can run truly in parallel on different cores.

3.4 Threading Models

User-Level Threads (ULT)

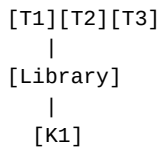
User-Level Threads are managed entirely in user space by a thread library (e.g., GNU Pth). The kernel sees only a single process — it is unaware of the threads. **Advantages:** very fast context switch (no system call); portable (no kernel changes). **Disadvantages:** if one thread makes a blocking system call, all threads in the process block; cannot run in parallel on multiple CPUs.

Kernel-Level Threads (KLT)

Kernel-Level Threads are managed directly by the kernel. Each thread has a kernel representation and can be scheduled independently. **Linux pthreads (NPTL)** use 1:1 mapping. **Advantages:** blocking by one thread doesn't block others; true parallelism on multi-core. **Disadvantages:** thread operations require system calls — slower than ULT.

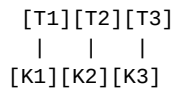
3.5 Multithreading Models

Many-to-One:



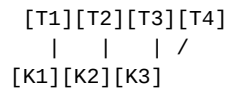
One user thread lib maps to ONE kernel thread. Blocking by one blocks all.

One-to-One:



Each user thread mapped to ONE kernel thread. True parallelism. High kernel overhead.

Many-to-Many:



M user threads → N kernel threads (M ≥ N). Maximum flexibility. Used by Windows thread pool, Solaris.

Model	Mapping	Trade-offs
Many-to-One	Multiple user threads → 1 kernel thread	Fast switch, no parallelism; blocking blocks all
One-to-One	1 user thread → 1 kernel thread	True parallelism; expensive — each thread is a kernel object
Many-to-Many	M user threads ↔ N kernel threads (M ≥ N)	Best of both; complex to implement; used in thread pools

3.6 Thread Issues

fork() and exec() in Multithreaded Programs

When a multithreaded process calls `fork()`, the child process gets only **one thread** — a copy of the thread that called `fork()`. Other threads are not copied. This can cause issues if those threads held locks (deadlock in child). If `exec()` is called right after `fork()` (common pattern), this is fine — `exec()` replaces the entire process image anyway. **POSIX:** after `fork()`, only async-signal-safe functions should be called in the child if `exec()` isn't immediately called.

Signal Handling

Signals can be delivered to specific threads or to the process. **Synchronous signals** (SIGSEGV, SIGFPE) are delivered to the thread that caused them. **Asynchronous signals** (SIGINT, SIGTERM) are delivered to an arbitrary thread or to a designated signal handler thread. POSIX provides `pthread_sigmask()` to control which threads accept which signals.

Thread Cancellation

Asynchronous cancellation: target thread is cancelled immediately — dangerous if thread holds locks or is in the middle of updating shared state. **Deferred cancellation (default):** target thread periodically checks for cancellation at cancellation points (`pthread_testcancel()`, I/O calls) and cancels only when safe.

QUICK REVISION BOX

- ✓ Thread = unit of execution within process; shares address space but has own PC, registers, stack.
- ✓ Threads vs Processes: threads share memory, faster to create/switch, but less fault-isolated.
- ✓ Multithreading benefits: Responsiveness, Resource sharing, Economy, Scalability.
- ✓ ULT: managed in user space — fast but no parallelism, blocking blocks all.

- ✓ KLT (Linux pthreads): kernel-managed — true parallelism, more overhead.
- ✓ Models: Many-to-One (fast, no parallel), One-to-One (Linux), Many-to-Many (thread pool).
- ✓ After `fork()`, only one thread exists in child — other threads not inherited.

CHAPTER 4: CPU Scheduling

4.1 Basic Concepts

CPU scheduling is the OS mechanism that decides which ready process gets the CPU next and for how long. The goal is to keep the CPU busy as much as possible while providing fair, responsive service. Processes alternate between **CPU bursts** (executing instructions) and **I/O bursts** (waiting for I/O). I/O-bound processes have many short CPU bursts; CPU-bound processes have few long CPU bursts. A good scheduler must handle both efficiently.

4.2 Scheduling Criteria

Criterion	Definition & Formula
CPU Utilisation	Percentage of time CPU is busy doing useful work. Target: 90–99%.
Throughput	Number of processes completed per unit time. More = better.
Turnaround Time (TAT)	Total time from submission to completion. $TAT = Completion - Arrival \text{ time}$.
Waiting Time (WT)	Time spent in Ready queue only. $WT = TAT - Burst \text{ time}$.
Response Time (RT)	Time from first submission to first response (important for interactive systems).

Formula summary: $TAT = Completion \text{ Time} - Arrival \text{ Time}$; $WT = TAT - Burst \text{ Time}$; $RT = First \text{ CPU Start} - Arrival \text{ Time}$; $Avg \text{ WT} = \sum(WT_i) / n$.

4.3 Preemptive vs Non-preemptive Scheduling

Aspect	Preemptive	Non-preemptive
Definition	Scheduler can forcibly remove CPU from running process	Process runs to completion once it gets CPU
Triggered by	Timer interrupt, higher-priority arrival	Only when process voluntarily yields (I/O, exit)
CPU Utilization	Better — no convoy effect	Can be poor if long jobs arrive first
Context Switch Rate	Higher (more switches)	Lower (fewer switches)
Response Time	Better for interactive/real-time	Worse — unpredictable for short jobs
Starvation Risk	Priority preemptive can starve low-priority	FCFS avoids starvation
Examples	SRTF, Round Robin, Priority (preemptive)	FCFS, SJF, Priority (non-preemptive)

4.4 Scheduling Algorithms

4.4.1 First-Come First-Serve (FCFS)

The simplest scheduling algorithm. Processes are dispatched in the order they arrive in the ready queue. **Non-preemptive.** Implemented as a simple FIFO queue. The main drawback is the **Convoy Effect**: if a CPU-bound process (large burst time) arrives first, all short I/O-bound processes must wait behind it — even processes that would take 1ms wait behind one that takes 10 seconds. This severely degrades throughput and responsiveness.

Example: 4 processes arrive at time 0. P1 burst=24, P2 burst=3, P3 burst=3, P4 burst=5.

FCFS Gantt Chart:



Waiting Times:

P1: 0 P2: 24 P3: 27 P4: 30

Turnaround Times:

P1: 24 P2: 27 P3: 30 P4: 35

Average Waiting Time = $(0+24+27+30)/4 = 81/4 = 20.25$ ms

Average Turnaround = $(24+27+30+35)/4 = 116/4 = 29.00$ ms

★ Convoy Effect: P2,P3,P4 (burst=3,3,5) wait 24+27+30ms behind P1 (burst=24)!

4.4.2 Shortest Job First (SJF) — Non-preemptive

SJF selects the process with the **smallest CPU burst time** from the ready queue. Non-preemptive: once a process starts, it runs to completion. SJF is **provably optimal** for minimising average waiting time (among non-preemptive algorithms). **Problem:** we cannot know future CPU burst lengths. In practice, we use **exponential averaging** to predict: $\tau_{n+1} = \alpha \times t_n + (1-\alpha) \times \tau_n$, where t_n is actual last burst, τ_n is previous estimate, $\alpha \in [0,1]$.

Example: P1 arr=0 burst=6; P2 arr=2 burst=4; P3 arr=4 burst=2; P4 arr=6 burst=3.

SJF (Non-preemptive) Gantt Chart:

At t=0: only P1 available → run P1 (burst=6)

At t=6: P2(4), P3(2), P4(3) all arrived → pick P3 (shortest, burst=2)

At t=8: P4(3), P2(4) → pick P4

At t=11: P2(4) → run P2



WT: P1=0, P2=9, P3=2, P4=5 → Avg WT = $(0+9+2+5)/4 = 4.00$ ms

TAT: P1=6, P2=13, P3=4, P4=8 → Avg TAT = $(6+13+4+8)/4 = 7.75$ ms

4.4.3 Shortest Remaining Time First (SRTF) — Preemptive SJF

SRTF is the preemptive version of SJF. Whenever a new process arrives, compare its burst time with the **remaining** burst of the currently running process. If the new process is shorter, preempt and run the new one. This gives the absolute minimum average waiting time among all scheduling algorithms (optimal).

Same processes: P1(arr=0,b=6), P2(arr=2,b=4), P3(arr=4,b=2), P4(arr=6,b=3)

t=0: P1 starts (only one). Remaining=6

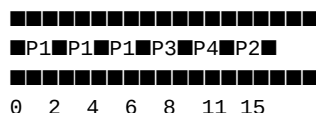
t=2: P2(4) arrives. P1 remaining=4, P2=4. Tie → P1 continues

t=4: P3(2) arrives. P1 remaining=2, P3=2. Tie → P1 continues

t=6: P4(3) arrives. P1 remaining=0! P1 finishes. P3(2), P2(4), P4(3) → pick P3

t=8: P4(3) vs P2(4). Pick P4

t=11: P2(4) runs to completion t=15



WT: P1=0, P2=7, P3=2, P4=5 → Avg WT = $(0+7+2+5)/4 = 3.50$ ms ← better than SJF!

TAT: P1=6, P2=11, P3=4, P4=8 → Avg TAT = $(6+11+4+8)/4 = 7.25$ ms

4.4.4 Priority Scheduling

Each process is assigned a **priority number**. The CPU is given to the process with the highest priority (convention: lower number = higher priority in most systems, though some use higher number = higher priority —

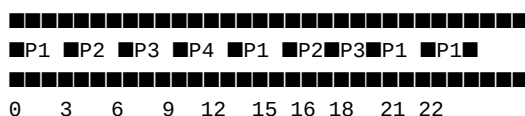
Example: P1(arr=0,b=10,pr=3), P2(arr=2,b=5,pr=1), P3(arr=4,b=3,pr=2), P4(arr=7,b=2,pr=4)
(Lower number = Higher priority)

```
t=0: Only P1. Run P1 (pr=3) until t=10
t=10: P2(pr=1),P3(pr=2),P4(pr=4) all arrived. Pick P2 (pr=1, highest)
t=15: P3(pr=2),P4(pr=4). Pick P3
t=18: P4. Run P4.
```



Round Robin is the most widely used scheduling algorithm for time-sharing systems. Each process gets a small fixed time unit called a **time quantum (q)** — typically 10–100ms. The ready queue is treated as a circular queue. After each quantum, the process is preempted and added to the tail of the ready queue. **If $q \rightarrow \infty$:** RR behaves like FCFS. **If $q \rightarrow 0$:** processor sharing — infinite context switch overhead. Rule of thumb: q should be large enough that 80% of CPU bursts are shorter than q (so most processes complete in one quantum).

```
t=0: [P1]→ run P1 (0-3), remaining=7. P2,P3,P4 arrived. Queue=[P2,P3,P4,P1]
t=3: Run P2 (3-6), remaining=1. Queue=[P3,P4,P1,P2]
t=6: Run P3 (6-9), remaining=2. Queue=[P4,P1,P2,P3]
t=9: Run P4 (9-12), remaining=0. P4 done. Queue=[P1,P2,P3]
t=12: Run P1 (12-15), remaining=4. Queue=[P2,P3,P1]
t=15: Run P2 (15-16), remaining=0. P2 done. Queue=[P3,P1]
t=16: Run P3 (16-18), remaining=0. P3 done. Queue=[P1]
t=18: Run P1 (18-21), remaining=1. Queue=[P1]
t=21: Run P1 (21-22). P1 done.
```



The ready queue is partitioned into multiple separate queues based on process type. Each queue has its own scheduling algorithm. Processes are **permanently assigned** to a queue. Higher-priority queues always preempt lower-priority ones. A common arrangement: Queue 1 (Interactive / foreground — RR); Queue 2 (Batch / background — FCFS); Queue 3 (Student / low priority — FCFS). **Problem:** starvation of lower queues if higher queues are always non-empty.

OS Placement Preparation Guide | Page 15 of 59

MLFQ improves on Multilevel Queue by allowing processes to **move between queues** based on their behaviour. New processes enter the highest-priority queue (short quantum). If a process uses its entire quantum without completing, it is **moved down** to the next lower queue (longer quantum). I/O-bound processes that frequently yield before their quantum expires stay in higher queues. After spending too long in low queues, a process is **boosted upward** (aging) to prevent starvation. MLFQ is the most general-purpose algorithm — Linux's CFS is inspired by it.

Q0 (q=4ms, RR) ■■■■■■■■■■ highest priority, new processes enter here
 ↓ used full quantum
 Q1 (q=8ms, RR) ■■■■■■■■■■ medium priority
 ↓ used full quantum
 Q2 (q=∞, FCFS) ■■■■■■■■■■ lowest priority
 ↑ (aging: after waiting too long → promote)

4.5 Scheduling Algorithm Comparison

Algorithm	Type	Advantage	Disadvantage	Use Case
FCFS	Non-preemptive	Simple, fair	Convoy effect, poor avg WT	Batch
SJF	Non-preemptive	Min avg WT (non-preemptive)	Starvation of long jobs; burst prediction	Batch
SRTF	Preemptive	Optimal avg WT	Starvation; burst prediction; overhead	Special cases
Priority	Both	Important tasks first	Starvation (fix: aging)	Real-time, OS tasks
Round Robin	Preemptive	Fair, good response time	High avg TAT; quantum tuning	Time-sharing, interactive
MLQ	Both	Separate policies per class	Starvation of lower queues	Desktop OS
MLFQ	Preemptive	Adaptive, no starvation (aging)	Complex to implement/tune	Linux, macOS, Windows

QUICK REVISION BOX

- ✓ Scheduling criteria: CPU Utilisation, Throughput, TAT, WT, Response Time.
- ✓ FCFS: simple but Convoy Effect kills throughput. SJF: optimal avg WT but needs future knowledge.
- ✓ SRTF: preemptive SJF — globally optimal avg WT; can cause starvation.
- ✓ Priority: efficient for critical tasks; aging prevents starvation.
- ✓ Round Robin: best for interactive/time-sharing; quantum size is critical.
- ✓ MLFQ: most practical — adapts to process behaviour; used in modern OSes.
- ✓ TAT = Completion – Arrival; WT = TAT – Burst; RT = First CPU Start – Arrival.

CHAPTER 5: Process Synchronization

5.1 Race Conditions

A **race condition** occurs when two or more processes or threads access shared data concurrently, and the final result depends on the relative timing of their execution — i.e., the outcome is non-deterministic. Race conditions are among the most insidious bugs: they may only manifest rarely under specific timing conditions, making them very hard to reproduce and debug.

```
/* Shared variable: counter = 0. Two threads run this concurrently. */
void increment() {
    counter++; // NOT atomic! Expands to 3 instructions:
               // (1) LOAD  R1 = counter    ← thread A reads 5
               // (2) ADD   R1 = R1 + 1     ← thread A computes 6
               //          ...thread B reads 5, computes 6, writes 6...
               // (3) STORE counter = R1    ← thread A writes 6
               //          RESULT: 6 instead of expected 7! ← lost update
}
```

5.2 Critical Section & Its Requirements

The **critical section** is the segment of code where a process accesses shared resources (shared variables, files, I/O devices). Only one process must be in its critical section at any time. A solution must satisfy all three of these requirements:

Requirement	Formal Definition
1. Mutual Exclusion	If process P_i is executing in its critical section, no other process can be executing in its critical section simultaneously.
2. Progress	If no process is in its critical section and some processes wish to enter it, the selection of the next entrant cannot be postponed indefinitely. Processes not in the remainder section participate in the decision.
3. Bounded Waiting	A bound (finite limit) must exist on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter its critical section. Prevents starvation.

```
/* Structure of a Critical Section solution */
do {
    /* ENTRY SECTION */ ← acquire permission to enter CS
    critical_section();
    /* EXIT SECTION */ ← signal departure from CS
    remainder_section();
} while (true);
```

5.3 Peterson's Solution

Peterson's solution is a classic software-based solution for the critical section problem for **two processes**. It uses two shared variables: `int turn` (whose turn it is to enter CS) and `bool flag[2]` (whether each process wants to enter CS).

```
/* Shared variables */
int  turn;           // turn=0: P0's turn; turn=1: P1's turn
bool flag[2];       // flag[i]=true: process i wants to enter CS

/* Process P0's entry/exit code (P1 is symmetric with 0↔1 swapped) */
flag[0] = true;      // I want to enter CS
turn = 1;            // But give priority to P1 first (polite)
while (flag[1] && turn == 1) // Busy-wait if P1 wants AND it's P1's turn
```

```

        ; // spin (busy wait)
// ---- CRITICAL SECTION ----
flag[0] = false; // I'm done, release CS

```

Correctness proof sketch: (1) **Mutual Exclusion:** both can only be in CS if both flag[]=true AND turn=both — impossible since turn is a single variable; (2) **Progress:** if P0 is waiting and P1 is not interested (flag[1]=false), P0's while condition is false — it enters; (3) **Bounded Waiting:** P0 will wait at most one turn of P1. **Limitation:** busy-waiting wastes CPU; works only for 2 processes; assumes atomic load/store (may not hold on modern out-of-order CPUs without memory barriers).

5.4 Hardware Synchronization

```

/* TestAndSet (TSL) – hardware atomic instruction */
bool TestAndSet(bool *target) {
    bool rv = *target; // read current value
    *target = true;    // atomically set to true
    return rv;        // return old value
}

/* Mutex lock using TestAndSet */
bool lock = false; // shared

void acquire() {
    while (TestAndSet(&lock)) // spin until lock was false (we set it true)
        ; // busy-wait
}

void release() { lock = false; }

/* Swap instruction (alternative) */
void Swap(bool *a, bool *b) {
    bool temp = *a; *a = *b; *b = temp; // atomic
}

```

5.5 Mutex Locks and Semaphores

Mutex Locks

A **mutex (mutual exclusion) lock** is a binary lock — it can be **locked** or **unlocked**. Only the thread that locked it can unlock it. The OS provides mutex operations: **acquire()** — blocks until lock is available, then takes it; **release()** — releases the lock, unblocks one waiting thread. Mutexes with busy-waiting are called **spinlocks** — expensive on single-core, but efficient on multi-core if critical sections are very short.

Semaphores

A **semaphore** is a more powerful synchronisation primitive invented by Dijkstra. It is an integer variable **S** with two atomic operations: **wait(S)** [P or down]: while $S \leq 0$ block; then $S--$; **signal(S)** [V or up]: $S++$; wake one waiting process. **Binary semaphore** (mutex): $S \in \{0,1\}$. **Counting semaphore:** S = number of available resources (e.g., 3 printers $\rightarrow S=3$).

```

/* Semaphore implementation with blocking (no busy wait) */
typedef struct {
    int value;
    struct process *list; // waiting processes
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add_to_list(S->list, current_process);
        block(); // suspend – no CPU waste
    }
}

```

```

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        P = remove_from_list(S->list);
        wakeup(P);
    }
}

```

5.6 Classical Synchronization Problems

5.6.1 Bounded-Buffer (Producer-Consumer) Problem

N buffers in a circular pool. Producers add items; consumers remove them. Three semaphores: **mutex=1** (binary — protects buffer), **empty=N** (counting — number of empty slots), **full=0** (counting — number of full slots).

```

semaphore mutex=1, empty=N, full=0;
int buffer[N]; int in=0, out=0;

/* PRODUCER */
void producer() {
    while (true) {
        item = produce_item();
        wait(empty);    // decrement empty slots; block if buffer full
        wait(mutex);    // enter critical section
        buffer[in] = item;
        in = (in + 1) % N;
        signal(mutex);  // leave critical section
        signal(full);   // increment full slots; wake consumer if waiting
    }
}

/* CONSUMER */
void consumer() {
    while (true) {
        wait(full);     // decrement full slots; block if buffer empty
        wait(mutex);    // enter critical section
        item = buffer[out];
        out = (out + 1) % N;
        signal(mutex);  // leave critical section
        signal(empty);  // increment empty slots; wake producer if waiting
        consume(item);
    }
}

```

■ INTERVIEW TIP: Order Matters!

In the Producer: wait(empty) MUST come before wait(mutex). Reversing them causes deadlock: if buffer is full, producer holds mutex while waiting on empty — consumer can never acquire mutex to consume.

5.6.2 Readers-Writers Problem

A shared database is read by many **readers** (concurrent reads are safe) and written by some **writers** (exclusive access needed). **First solution (readers' priority)**: readers never wait unless a writer has already been granted access. Writers may starve if readers keep arriving.

```

semaphore rw_mutex=1, mutex=1;    // rw_mutex: writer/reader exclusion; mutex: protects read_count
int read_count = 0;

/* WRITER */
void writer() {
    wait(rw_mutex);                // exclusive access
    /* ... write ... */
}

```

```

        signal(rw_mutex);
    }

/* READER (First solution – readers priority) */
void reader() {
    wait(mutex);
    read_count++;
    if (read_count == 1) wait(rw_mutex); // first reader locks out writers
    signal(mutex);
    /* ... read ... */
    wait(mutex);
    read_count--;
    if (read_count == 0) signal(rw_mutex); // last reader unlocks writers
    signal(mutex);
}

```

5.6.3 Dining Philosophers Problem

Five philosophers sit at a circular table. Each needs two forks (left and right) to eat. If all pick up left fork simultaneously — **deadlock** (each waits for right). **Solution 1 — Asymmetric:** philosophers 0-3 pick left then right; philosopher 4 picks right then left (breaks circular wait). **Solution 2 — Allow only 4 philosophers at table simultaneously** (counting semaphore table=4). **Solution 3 — Monitor with states:** only pick forks when both are available.

```

semaphore fork[5] = {1,1,1,1,1}; // one semaphore per fork */
semaphore table = 4; // at most 4 philosophers at table */

void philosopher(int i) {
    while (true) {
        think();
        wait(table); // sit down only if < 4 already seated
        wait(fork[i]); // pick up left fork
        wait(fork[(i+1)%5]); // pick up right fork
        eat();
        signal(fork[i]); // put down left fork
        signal(fork[(i+1)%5]); // put down right fork
        signal(table); // leave table
    }
}

```

5.7 Monitors

A **monitor** is a high-level synchronisation construct — an abstract data type that encapsulates shared data and operations on it. The monitor guarantees that only one process can be active inside the monitor at any time (mutual exclusion is built-in, unlike semaphores where you must manually call wait/signal). **Condition variables** allow processes to wait inside the monitor: `x.wait()` — suspends calling process; `x.signal()` — resumes one suspended process. Java's synchronized methods implement a monitor.

QUICK REVISION BOX

- ✓ Race condition: non-deterministic result from unsynchronized concurrent access to shared data.
- ✓ Critical Section: code accessing shared resources. Must satisfy: Mutual Exclusion, Progress, Bounded Waiting.
- ✓ Peterson's Solution: software-only for 2 processes; uses turn + flag[]; susceptible to busy-waiting.
- ✓ TestAndSet/Swap: hardware atomic instructions enabling spinlocks.
- ✓ Semaphore: wait(S) decrements & blocks if negative; signal(S) increments & wakes. Binary for mutex, counting for resource pooling.

- ✓ Producer-Consumer: mutex + empty(N) + full(0); order wait(empty) before wait(mutex) in producer.
- ✓ Dining Philosophers: deadlock if all pick left fork. Fix: asymmetric pickup, or limit to 4 philosophers.

CHAPTER 6: Deadlocks

6.1 What is a Deadlock?

A **deadlock** is a situation where a set of processes are each waiting for a resource held by another process in the set, and no process can make progress. Think of two single-lane roads merging: Car A going north blocks Car B going east, and Car B blocks Car A. Neither can move. In computing, processes hold resources (locks, memory, I/O devices) while requesting others, leading to circular dependency.

6.2 Coffman's Four Necessary Conditions

A deadlock can occur **if and only if** all four of the following conditions hold simultaneously. Attacking any one of them prevents deadlock.

Condition	Definition	How to Eliminate
1. Mutual Exclusion	At least one resource must be held in a non-sharable mode — only one process can use it at a time.	Allow sharing (not always possible for printers, locks)
2. Hold and Wait	A process is holding at least one resource and is waiting to acquire additional resources held by other processes.	Require processes to request all resources at once (low utilisation) or release all before requesting new ones
3. No Preemption	Resources cannot be forcibly taken from a process; only voluntarily released.	Allow preemption — if a process can't get all resources, release what it has (works for CPU, memory; not for printers)
4. Circular Wait	There exists a set {P0, P1, ..., Pn} where P0 waits for P1, P1 waits for P2, ..., Pn waits for P0.	Impose a total ordering on all resource types; processes must request in increasing order (R1 before R2 before R3)

6.3 Resource Allocation Graph (RAG)

A **Resource Allocation Graph** is a directed graph used to detect deadlocks. **Nodes:** circles P = processes; rectangles R = resource types (with dots inside for instances). **Edges:** **Request edge** P→R (P wants R); **Assignment edge** R→P (R is held by P). **Rule:** if graph has no cycle → no deadlock. If cycle exists: if each resource type has exactly one instance → deadlock; if multiple instances → cycle is necessary but not sufficient for deadlock.

RAG Example – DEADLOCK (single instances):

P1 ■■request■■■ R1

R1 ■■assigned■■ P2

P2 ■■request■■■ R2

R2 ■■assigned■■ P1

Cycle: P1→R1→P2→R2→P1 → DEADLOCK ✗

RAG Example – NO DEADLOCK (despite cycle, multiple instances):

R1 has 2 instances. P1 holds R1, requests R2.

P2 holds R1 and R2, requests R1.

P3 holds R2.

Cycle: P1→R2→P2→R1→P1 and P2→R1→P3→R2→P2

But P3 can finish, release R2 → P1 gets R2 → P1 finishes → P2 gets resources. No deadlock ✓

6.4 Banker's Algorithm (Deadlock Avoidance)

The **Banker's Algorithm** (Dijkstra, 1965) is a deadlock avoidance algorithm. When a process requests resources, the OS simulates granting the request and checks if the resulting state is **safe**. If safe, grant; if unsafe,

block the request. A state is **safe** if there exists a **safe sequence** — an ordering of all processes such that each process can eventually obtain all needed resources, run to completion, and release resources for the next.

Data Structures

$n = 5$ processes ($P_0..P_4$), $m = 3$ resource types (A,B,C)

Available: [3, 3, 2] ← currently available instances

Max demand: Allocation (currently held):

	A	B	C		A	B	C
P0:	[7,	5,	3]	P0:	[0,	1,	0]
P1:	[3,	2,	2]	P1:	[2,	0,	0]
P2:	[9,	0,	2]	P2:	[3,	0,	2]
P3:	[2,	2,	2]	P3:	[2,	1,	1]
P4:	[4,	3,	3]	P4:	[0,	0,	2]

Need = Max – Allocation:

P0: [7,4,3] P1:[1,2,2] P2:[6,0,0] P3:[0,1,1] P4:[4,3,1]

Safety Algorithm — Worked Example

SAFETY ALGORITHM:

Work = Available = [3,3,2]; Finish[i] = false for all i

STEP 1: Find i where Finish[i]=false AND Need[i] ≤ Work

P1: Need=[1,2,2] ≤ Work=[3,3,2]? YES → run P1

Work = Work + Allocation[P1] = [3,3,2]+[2,0,0] = [5,3,2]

Finish[P1]=true

STEP 2: Find next i. Try P3: Need=[0,1,1] ≤ [5,3,2]? YES → run P3

Work = [5,3,2]+[2,1,1] = [7,4,3]; Finish[P3]=true

STEP 3: Try P4: Need=[4,3,1] ≤ [7,4,3]? YES → run P4

Work = [7,4,3]+[0,0,2] = [7,4,5]; Finish[P4]=true

STEP 4: Try P0: Need=[7,4,3] ≤ [7,4,5]? YES → run P0

Work = [7,4,5]+[0,1,0] = [7,5,5]; Finish[P0]=true

STEP 5: Try P2: Need=[6,0,0] ≤ [7,5,5]? YES → run P2

Work = [7,5,5]+[3,0,2] = [10,5,7]; Finish[P2]=true

All Finish[i]=true → SAFE SEQUENCE: <P1, P3, P4, P0, P2> ✓ STATE IS SAFE

Resource-Request Algorithm

RESOURCE-REQUEST: P1 requests [1,0,2]

Step 1: Check Request[1] ≤ Need[1]: [1,0,2] ≤ [1,2,2]? YES ✓

Step 2: Check Request[1] ≤ Available: [1,0,2] ≤ [3,3,2]? YES ✓

Step 3: Pretend to allocate:

Available = [3,3,2] - [1,0,2] = [2,3,0]

Allocation[1] = [2,0,0] + [1,0,2] = [3,0,2]

Need[1] = [1,2,2] - [1,0,2] = [0,2,0]

Step 4: Run safety algorithm on new state → if SAFE → grant request ✓

if UNSAFE → restore old state, block P1's request

6.5 Deadlock Detection & Recovery

Detection (single instance): Maintain a **Wait-For Graph** (WFG) — derived from RAG by removing resource nodes. Edge $P \rightarrow Q$ means P is waiting for Q. A cycle in WFG means deadlock. Invoke cycle detection periodically ($O(n^2)$).

Recovery: (1) **Process termination:** abort all deadlocked processes (safe, drastic) or abort one at a time until deadlock is broken (expensive — detection after each abort). (2) **Resource preemption:** select a victim (minimise cost), rollback victim to safe state, reallocate its resources. Must avoid same process always being victim (starvation) — include rollback count in cost function.

6.6 Ostrich Algorithm

Most general-purpose operating systems (Linux, Windows, macOS) use the **Ostrich Algorithm**: simply ignore the deadlock problem and pretend it doesn't happen. Rationale: deadlocks are infrequent, and the overhead of prevention/avoidance would hurt performance more than the occasional deadlock. Users just restart the hung process. This is pragmatically the most common approach in practice, though it is philosophically inelegant.

QUICK REVISION BOX

- ✓ Deadlock: circular waiting where no process can proceed. Real-world analogy: traffic gridlock.
- ✓ Four Coffman conditions (ALL must hold): Mutual Exclusion, Hold-and-Wait, No Preemption, Circular Wait.
- ✓ Elimination strategy: attack any ONE condition (easiest: eliminate circular wait via resource ordering).
- ✓ RAG: cycle + single instances → guaranteed deadlock. Multiple instances: cycle is necessary, not sufficient.
- ✓ Banker's Algorithm: check if allocation leads to safe state before granting. $\text{Need} = \text{Max} - \text{Allocation}$.
- ✓ Safe state → safe sequence exists where every process can eventually finish.
- ✓ Detection: Wait-For Graph for single instances. Recovery: terminate process or preempt resources.
- ✓ Ostrich Algorithm: Linux/UNIX practical approach — ignore rare deadlocks.

CHAPTER 7: Memory Management — Basics

7.1 Memory Hierarchy

Level	Access Time	Typical Capacity	Cost/Byte	Notes
CPU Registers	< 1 ns	~1 KB	Highest	In CPU — fastest, most expensive
L1/L2/L3 Cache	1–10 ns	256KB–32MB	Very High	On/near CPU die; SRAM
Main Memory (RAM)	60–100 ns	4GB–1TB	Medium	DRAM; primary program storage
SSD (NVMe)	10–100 µs	256GB–8TB	Low	Persistent; much faster than HDD
HDD	5–15 ms	1TB–20TB	Very Low	Persistent; cheapest per GB

7.2 Address Binding

Before a program executes, its symbolic addresses (variable names, function names) must be bound to actual physical memory addresses. This binding can occur at three stages:

Binding Stage	Mechanism & Limitation	Example
Compile Time	Absolute code generated — must know exact load address. If address changes, must recompile.	MS-DOS .COM files
Load Time	Relocatable code generated; binding done when loaded into memory. Cannot be moved after loading.	Early Unix static linking
Execution Time	Addresses translated dynamically by MMU at runtime. Process can be moved in memory while running.	Modern OSes with virtual memory

7.3 Logical vs Physical Address Space

A **logical (virtual) address** is generated by the CPU during program execution — it is what the program 'sees'. A **physical address** is the actual address in main memory hardware (RAM chips). The distinction enables: (1) processes to believe they have a large, contiguous address space even when physical memory is fragmented; (2) the OS to move processes in physical memory without the program knowing; (3) memory protection between processes.

The **Memory Management Unit (MMU)** is hardware that translates logical to physical addresses at runtime. The simplest MMU uses a **relocation register**: $\text{Physical Address} = \text{Logical Address} + \text{Relocation Register}$. The **limit register** ensures the logical address doesn't exceed the process's allocated size.

```
Process A sees:  logical addr 0x0000 → 0x3FFF  (16KB)
OS maps it to:   physical addr 0x8000 → 0xBFFF  (16KB starting at 32KB)
```

MMU computation:

```
If logical_addr < limit_register: physical_addr = logical_addr + base_register
Else: generate SEGMENTATION FAULT (protection violation trap)
```

7.4 Contiguous Memory Allocation

Fixed Partitions

Memory divided into fixed-size partitions at system start. Each partition holds one process. **Internal fragmentation**: process smaller than partition — wasted space inside the partition. Degree of multiprogramming is limited to number of partitions.

Variable Partitions

Partitions sized exactly to each process. When a process finishes, it leaves a hole. Over time, memory becomes a patchwork of processes and holes — **external fragmentation**: total free memory is sufficient but not contiguous. **Compaction** (defragmentation) moves all processes to one end — expensive. Only feasible with execution-time address binding.

Strategy	Description	Advantage	Disadvantage
First Fit	Allocate first hole large enough. Scan from beginning.	Fast; simple	Tends to fragment start of list
Best Fit	Allocate smallest hole that's large enough. Scan entire list.	Minimises wasted space per allocation	Slow; leaves tiny unusable holes
Worst Fit	Allocate largest hole. Scan entire list.	Leaves largest remnant (usable)	Slow; destroys large holes quickly
Next Fit	Like First Fit but resumes from where last search ended.	Fast; distributes allocations evenly	Slightly more fragmentation than First Fit

7.5 Swapping

Swapping is the process of moving an entire process temporarily from main memory to a **swap space** (backing store — typically a disk partition or file) to free up RAM for other processes. The process is **rolled out** to disk and later **rolled in** back to memory. The major disadvantage is the high latency — swapping a 100MB process to disk at 50MB/s takes 2 seconds. Modern systems use **demand paging** instead of full swapping, but swapping still occurs in memory pressure.

QUICK REVISION BOX

- ✓ Memory hierarchy: Registers (fastest) → Cache → RAM → SSD → HDD (slowest, cheapest).
- ✓ Address binding: Compile-time (absolute) → Load-time (relocatable) → Execution-time (virtual; modern OSes).
- ✓ Logical address ≠ Physical address. MMU translates at runtime using base+limit registers.
- ✓ Fixed partitions → internal fragmentation. Variable partitions → external fragmentation.
- ✓ Allocation strategies: First Fit (fast), Best Fit (min waste), Worst Fit, Next Fit.
- ✓ External fragmentation solution: compaction (expensive) or virtual memory / paging.
- ✓ Swapping: move whole process to disk (roll-out) and back (roll-in); very slow.

CHAPTER 8: Paging & Segmentation

8.1 Paging

Paging is the most widely used memory management scheme. It eliminates **external fragmentation** by using fixed-size units for both physical memory (**frames**) and logical memory (**pages**). Physical memory is divided into equal-sized blocks called **frames**. Logical memory is divided into blocks of the same size called **pages**. A page table maps page numbers to frame numbers. Typical page sizes: 4KB, 2MB (hugepages).

Address Translation:

Logical Address = [Page Number (p) | Page Offset (d)]

Example: 32-bit address, 4KB pages (12-bit offset)

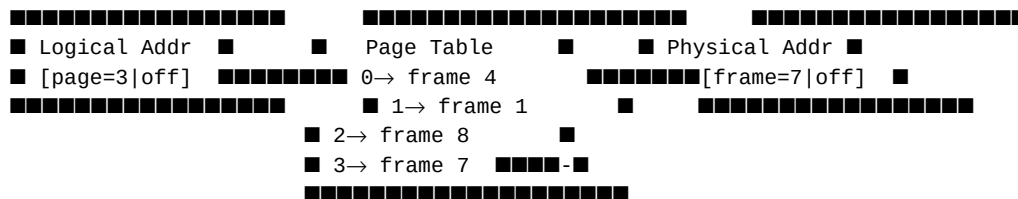
Logical addr 0x0000_3500:

p = 0x0000_3500 >> 12 = 0x3 (page 3)

d = 0x0000_3500 & 0xFFF = 0x500 (offset 1280)

Page Table lookup: page_table[3] = frame 7

Physical addr = (7 << 12) | 0x500 = 0x0000_7500



8.2 TLB and Effective Access Time

Without hardware support, each memory access requires two accesses: one for the page table (to find frame), one for actual data. This doubles memory access time. The **Translation Lookaside Buffer (TLB)** is a small, fast fully-associative hardware cache inside the CPU's MMU that caches recent page→frame translations. TLB hit: single-cycle lookup. TLB miss: page table walk.

Effective Access Time (EAT) Formula:

$$EAT = (\text{hit_ratio} \times (\text{TLB_time} + \text{memory_time})) + ((1 - \text{hit_ratio}) \times (\text{TLB_time} + 2 \times \text{memory_time}))$$

Example: TLB time = 10ns, Memory time = 100ns, Hit ratio $\alpha = 0.95$

$$EAT = 0.95 \times (10+100) + 0.05 \times (10+200)$$
$$= 0.95 \times 110 + 0.05 \times 210$$
$$= 104.5 + 10.5 = 115 \text{ ns} \quad (\text{vs } 210 \text{ ns without TLB: } \sim 45\% \text{ faster})$$

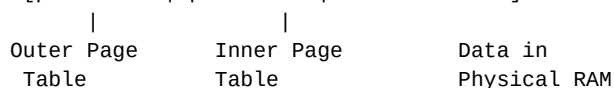
Modern TLBs: 64-1024 entries, ~99% hit ratio → EAT ≈ 101-102 ns

8.3 Multi-level Paging

A single-level page table for a 32-bit address space with 4KB pages has $2^{32} = 1,048,576$ entries × 4 bytes = 4MB per process, always resident in RAM. For a 64-bit address space: impossibly large. **Multi-level paging** uses a hierarchy of page tables — only the portions actually in use need to be resident. In a **two-level page table** (32-bit, 4KB pages): logical address = [p1 (10 bits) | p2 (10 bits) | offset (12 bits)]. p1 indexes the outer page table; p2 indexes the inner page table for that subtree.

Two-Level Page Table (32-bit):

Logical: [p1=10bits | p2=10bits | offset=12bits]





Only inner page tables for USED regions need to exist in RAM → huge savings.

64-bit Linux: 4-level page table (PGD→PUD→PMD→PTE)

64-bit Linux 5.x: 5-level page table (PGD→P4D→PUD→PMD→PTE)

8.4 Segmentation

Segmentation divides logical memory into variable-size segments corresponding to logical units of a program: code segment, data segment, stack, heap, symbol table, etc. Unlike paging (transparent to programmer), segmentation **matches programmer's view** of memory. A logical address is a pair **<segment number, offset>**.

Segment Table: each entry has (base, limit)

Logical addr <segment=2, offset=100>:

seg_table[2] = {base=4300, limit=400}

Check: offset(100) < limit(400)? YES → physical = 4300+100 = 4400 ✓

If offset ≥ limit → SEGMENTATION FAULT (trap to OS)

8.5 Paging vs Segmentation

Aspect	Paging	Segmentation
Size of units	Fixed-size (4KB, 2MB)	Variable-size (logical units)
Fragmentation	Internal fragmentation only	External fragmentation
Programmer visibility	Transparent to programmer	Visible — programmer defines segments
Sharing support	Easy to share individual pages	Easy to share whole segments (code segment)
Protection	Per-page protection bits (R/W/X/valid)	Per-segment protection (read-only code)
Table size	Potentially large (one entry per page)	Small (one entry per segment)
Compaction	Not needed (frames can be non-contiguous)	Needed to combat external fragmentation
Real-world use	Used in all modern OSes (Linux, Windows)	Used in combination with paging (Intel IA-32)

QUICK REVISION BOX

- ✓ Paging: fixed-size pages (logical) ↔ frames (physical). Eliminates external fragmentation.
- ✓ Page table maps page numbers to frame numbers. TLB caches recent mappings.
- ✓ $EAT = \alpha(TLB + mem) + (1 - \alpha)(TLB + 2 \times mem)$. Modern hit ratio ~99% makes paging practical.
- ✓ Multi-level paging: saves memory by only allocating page tables for used regions.
- ✓ Segmentation: variable-size logical units; matches programmer view; causes external fragmentation.
- ✓ Modern systems combine paging+segmentation: segments are paged (IA-32, Linux on x86).
- ✓ Paging = internal fragmentation (max 1 page per process). Segmentation = external fragmentation.

Belady's Anomaly: Reference 1 2 3 4 1 2 5 1 2 3 4 5
 3 frames: 9 faults ← MORE faults with fewer frames (expected)
 4 frames: 10 faults ← ANOMALY: more frames, MORE faults!

OPT replaces the page that will **not be used for the longest time in the future**. This is provably optimal — gives the minimum number of page faults for any algorithm. **Cannot be implemented in practice** (requires future knowledge) but serves as a benchmark. Free from Bélády's Anomaly.

LRU replaces the page that was **least recently used** — it uses past access history to approximate future behaviour (locality of reference). LRU has the **stack property**: a set of pages for n frames is always a subset of the set for $n+1$ frames $\rightarrow$ no Bélády's Anomaly. Implementations: (1) **Counter**: timestamp each access, evict smallest; (2) **Stack**: move page to top on access, evict bottom.

A hardware-efficient approximation of LRU using a single **reference bit**. When a page is accessed, its reference bit is set to 1. When the OS needs a victim, it scans pages in FIFO order: if reference bit = 1, clear it (second chance, keep page) and advance; if reference bit = 0, evict this page. The clock hand wraps around in a circular buffer. If all pages have reference bit = 1, the algorithm degenerates to FIFO after one full scan.

Algorithm	Policy	Advantage	Disadvantage
FIFO	Evict oldest page	Simple, $O(1)$	Bélády's Anomaly; may evict frequent pages
OPT	Evict farthest future use	Minimum page faults	Not implementable (needs future)
LRU	Evict least recently used	No Bélády's; good in practice	Expensive: counter/stack overhead
Clock (2nd chance)	FIFO + reference bit	Near-LRU performance; hardware-efficient	Slightly worse than true LRU
LFU	Evict least frequently used	Good for stable hot set	New pages always evicted first

9.5 Thrashing

Thrashing occurs when a process does not have enough frames to support its **working set**. The process spends more time paging (page faults → disk I/O) than executing instructions. As degree of multiprogramming increases beyond a threshold, CPU utilisation drops catastrophically because all processes are waiting for their pages to be brought in from disk.

Working Set Model (Denning, 1968): the working set $W(t, \Delta)$ is the set of pages referenced in the last Δ time units (the **working set window**). The process needs at least $|W(t, \Delta)|$ frames to avoid thrashing. The OS tracks working sets and only admits new processes when sum of working sets $\leq$ total frames. **Page Fault Frequency (PFF):** monitor the rate of page faults per process. If rate too high → allocate more frames. If rate too low → reclaim frames.

QUICK REVISION BOX

- ✓ Virtual memory: process sees large address space regardless of physical RAM size.
- ✓ Demand paging: pages loaded only on access. Page fault → OS finds frame → disk read → restart instruction.
- ✓ COW: `fork()` shares pages as read-only; copy only on write → fast fork.
- ✓ FIFO: simple, Bélády's Anomaly possible. OPT: optimal but unimplementable. LRU: best practical.
- ✓ LRU has stack property → no Bélády's Anomaly. Clock algorithm approximates LRU efficiently.
- ✓ Thrashing: process needs more frames than allocated → constant page faults → near-zero CPU utilisation.
- ✓ Working Set Model prevents thrashing by ensuring each process has its active pages in memory.

CHAPTER 10: File Systems

10.1 File Concept and Attributes

A **file** is a named collection of related information that is recorded on secondary storage. From the OS perspective, a file is the smallest logical unit of secondary storage. Files abstract the physical details of the storage device — programmers deal with filenames and offsets, not disk sectors.

Attribute	Description
Name	Human-readable symbolic identifier (only form kept in human-readable form)
Identifier	Unique non-human-readable number (inode number in Unix) identifying the file within the file system
Type	Extension or magic bytes: .txt, .exe, .mp4 — helps OS determine how to process the file
Location	Pointer to file location on device (block addresses)
Size	Current size in bytes (and perhaps maximum allowed size)
Protection	Access control information — who can read, write, execute
Time/Date/User	Creation time, last modification time, last access time, owner user ID

10.2 Directory Structures

Structure	Description & Trade-offs
Single-Level	All files in one directory. Simple but name conflicts — can't have same name twice. Only works for single user.
Two-Level	Separate directory per user (master file directory + user file directories). Solves name conflicts. Poor for collaboration.
Tree-Structured	Generalized hierarchy. Current directory + absolute/relative paths. Most common (Unix, Windows NTFS).
Acyclic-Graph	Allows shared files/directories via hard links (inode sharing) or symbolic links. Need reference counting to delete safely.
General Graph	Cycles allowed via symbolic links. Risk of infinite loops during traversal — needs garbage collection.

10.3 File Allocation Methods

10.3.1 Contiguous Allocation

Each file occupies a **contiguous set of blocks** on disk. Directory entry stores start block and length. **Advantages:** Excellent sequential and random access — $O(1)$ seek to block b of file starting at block S : seek to $S+b$. Simple. **Disadvantages:** External fragmentation (holes between files); file cannot grow easily (next blocks may be allocated). Requires compaction occasionally.

10.3.2 Linked Allocation (& FAT)

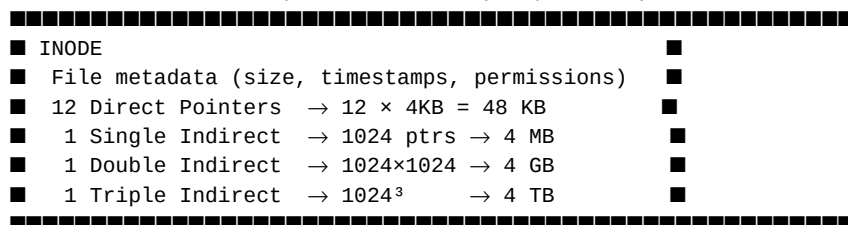
File is a **linked list of disk blocks** — each block has a pointer to the next. Directory entry stores only first and last block pointers. **Advantages:** No external fragmentation; file can grow easily — allocate any free block and link it. **Disadvantages:** No efficient random access (must traverse from start); pointer space overhead; disk failures corrupt links. **FAT (File Allocation Table):** stores all pointers in a table at disk start (not in blocks)

themselves) — enables faster access since FAT can be cached in RAM.

10.3.3 Indexed Allocation (UNIX Inodes)

Each file has an **index block** containing pointers to all data blocks. Supports efficient random access. No external fragmentation. UNIX uses a sophisticated **inode structure** combining direct, single-indirect, double-indirect, and triple-indirect blocks to support small and large files efficiently.

UNIX Inode Structure (4KB blocks, 4-byte pointers):



File Access Pattern:

Small files ($\leq 48\text{KB}$): 1-2 disk accesses (inode + direct block)

Medium files: inode + indirect block + data block (3 accesses)

Large files: up to 4 disk accesses (inode + 3 levels indirect)

Method	Advantages	Disadvantages	Used In
Contiguous	Simple, fast random access	External fragmentation; can't grow file	CD-ROMs, DVDs
Linked	No ext. fragmentation; dynamic growth	No random access; pointer overhead; fragile	FAT32 (Windows old)
Indexed (Inode)	Fast random access; no ext. fragmentation	Index block overhead; complex multi-level	ext4 (Linux), HFS+ (macOS)

10.4 Hard Links vs Soft Links

Aspect	Hard Link	Symbolic (Soft) Link
Mechanism	Directory entry pointing to same inode	Directory entry containing path string to target
Inode	Same inode as original (reference count++)	Own inode; stores target path
Cross-filesystem	NO — must be on same filesystem	YES — can span filesystems
Broken link?	Never — file exists as long as link count > 0	Yes — dangling pointer if target deleted
ls -la	Shows same inode number	Shows l (symlink) in permissions
Commands	ln file hardlink	ln -s file softlink

10.5 Free Space Management

Method	Mechanism	Used By
Bit Vector (Bitmap)	One bit per block: 0=free, 1=allocated. Easy to find contiguous free blocks. Bitmap for 1TB disk, 4KB blocks = 32MB — must be kept in memory.	Ext2/ext4
Linked List	Free blocks linked together; pointer in each free block to next. No wasted space for bitmap, but traversal slow.	Early UNIX
Grouping	First free block stores addresses of n free blocks; last stores address of next group. Faster than linked list.	BSD UFS
Counting	Store (first_free_block, count) pairs. Efficient when free blocks are contiguous runs.	NTFS

QUICK REVISION BOX

- ✓ File = named collection of data on secondary storage with metadata (name, size, permissions, timestamps).
- ✓ Directory structures: Single-Level → Two-Level → Tree (most common) → Acyclic Graph → General Graph.
- ✓ Contiguous allocation: fast random access but external fragmentation and can't grow files.
- ✓ Linked allocation: flexible growth, no external fragmentation, but $O(n)$ random access. FAT caches links in RAM.
- ✓ Indexed (inode) allocation: efficient random access + growth. UNIX inode uses direct+indirect+double+triple pointers.
- ✓ Hard link: same inode, no cross-filesystem. Soft link: path reference, cross-filesystem, can dangle.
- ✓ Free space: Bitmap (fast contiguous search) or Linked list or Grouping or Counting.

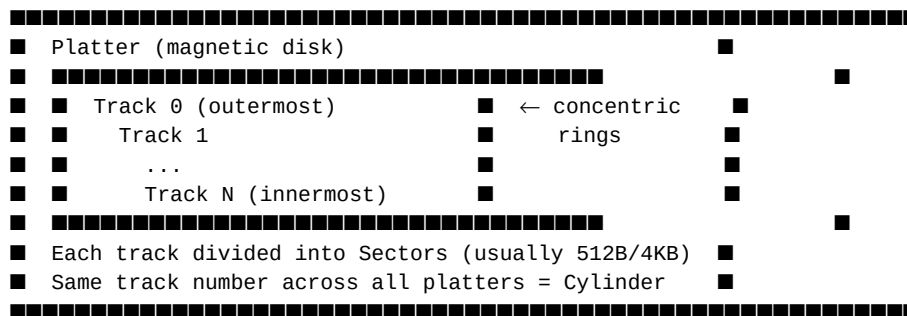
CHAPTER 11: I/O Systems & Disk Management

11.1 I/O Hardware and Techniques

Technique	Mechanism	Use Case
Programmed I/O (Polling)	CPU continuously polls device status register until I/O complete. Simple but wastes 100% CPU during I/O wait.	Low-speed devices, embedded systems
Interrupt-driven I/O	CPU initiates I/O, then continues other work. Device interrupts CPU when done (Interrupt Service Routine handles completion).	Most I/O in modern OS
DMA (Direct Memory Access)	DMA controller transfers data directly between device and RAM without CPU involvement. CPU only interrupted at start and end of transfer.	Disk, NIC, GPU, high-bandwidth devices

11.2 Disk Structure and Timing

DISK ANATOMY:



Access Time = Seek Time + Rotational Latency + Transfer Time
Seek Time: time to move head to correct track (5-15ms typical)
Rotational Latency: time for sector to rotate under head (avg = 1/2 rotation)
At 7200 RPM: full rotation=8.3ms, avg latency=4.2ms
Transfer Time: time to read/write data (small fraction for one sector)

11.3 Disk Scheduling Algorithms

Goal: minimise total head movement (seek time). **Request queue:** 98, 183, 37, 122, 14, 124, 65, 67. Head initially at track **53**. Total tracks: 0-199.

Request queue: 98, 183, 37, 122, 14, 124, 65, 67 Head: 53

FCFS (serve in arrival order):

53→98→183→37→122→14→124→65→67

Movement: 45+85+146+85+108+110+59+2 = 640 cylinders

SSTF (shortest seek time first):

53→65→67→37→14→98→122→124→183

Movement: 12+2+30+23+84+24+2+59 = 236 cylinders

★ Starvation possible: requests near center always served first

SCAN (Elevator – moves in one direction, then reverses):

53→65→67→98→122→124→183→(end 199)→37→14

Movement: (199-53) + (199-14) = 146+185 = 331 cylinders

Note: head moves to end even if no requests there

C-SCAN (Circular SCAN – go to end, jump to 0, scan again):

53→65→67→98→122→124→183→199(end)→0→14→37
 Movement: $(199-53) + (199-0) + 37 = 146+199+37 = 382$ cylinders
 Advantage: more uniform wait time than SCAN

LOOK (like SCAN but only goes to last request, not disk end):
 53→65→67→98→122→124→183→37→14
 Movement: $(183-53) + (183-14) = 130+169 = 299$ cylinders

C-LOOK (like C-SCAN but only to last request):
 53→65→67→98→122→124→183→14→37
 Movement: $(183-53) + (183-14) + (37-14) = 130+169+23 = 322$ cylinders

Algorithm	Policy	Advantage	Disadvantage
FCFS	Arrival order	Simple, fair	Worst head movement; no optimisation
SSTF	Nearest request	Best throughput (short seeks)	Starvation of far requests
SCAN (Elevator)	Direction + scan	No starvation; good throughput	Cylinder 0/N hit even with no requests there
C-SCAN	One direction, jump to start	Uniform wait times	Slightly more movement than SCAN
LOOK	Direction to last request	Avoids unnecessary travel	Common in Linux (deadline I/O scheduler)
C-LOOK	LOOK + circular	Best balance of uniform wait + performance	Most commonly recommended

11.4 RAID Levels

Level	Mechanism	Advantage	Disadvantage
RAID 0	Striping (no redundancy)	Max performance, max capacity	No fault tolerance — one failure loses all
RAID 1	Mirroring	Full redundancy; fast reads	50% capacity overhead; writes to both disks
RAID 5	Block-level striping + distributed parity	Good balance performance/reliability; 1 disk overhead	Write penalty (read-modify-write parity); slow rebuild
RAID 6	RAID 5 + second parity block (P+Q)	Survives 2 simultaneous disk failures	2-disk overhead; slower writes than RAID 5
RAID 10	RAID 1+0: mirror then stripe	High performance + redundancy	50% capacity overhead; expensive

QUICK REVISION BOX

- ✓ I/O techniques: Polling (CPU busy-waits), Interrupt-driven (CPU notified on completion), DMA (CPU-free bulk transfer).
- ✓ Disk access time = Seek time (dominant) + Rotational latency + Transfer time.
- ✓ SSTF minimises head movement but causes starvation. LOOK/C-LOOK best balance.
- ✓ SCAN = elevator algorithm; C-SCAN provides more uniform wait times.
- ✓ RAID 0: performance, no redundancy. RAID 1: full mirror. RAID 5: distributed parity, 1-disk loss tolerated.

- ✓ RAID 10 = RAID 1+0: best performance+redundancy combination at 50% capacity cost.
- ✓ Buffering: single/double/circular; Spooling: printer queue — device serves one process but seems to serve many.

CHAPTER 12: Linux & System Programming Basics

12.1 Linux File Permissions

```
ls -la output:
-rwxr-xr--  2 amrit developers 4096 Jan 1 12:00 script.sh
■■■■■ ■ ■
■■■■■■■ ■■ Others: r--(4) = read only
■■■■■■■■■■ Group: r-x(5) = read+execute
■■■■■■■■■■ Owner: rwx(7) = read+write+execute
■■■■■■■■■■ File type: -(regular), d(dir), l(link)
```

```
chmod 755 file    → rwxr-xr-x (owner: 7=rwx, group: 5=r-x, other: 5=r-x)
chmod u+x file    → add execute for user
umask 022         → default: files=644, dirs=755
```

Special Bits:

SUID (4000): executes with file OWNER's UID (/usr/bin/passwd → runs as root)

SGID (2000): executes with file GROUP's GID; on dir: new files inherit dir's group

Sticky(1000): on directory, only file owner can delete their files (/tmp has sticky bit)

12.2 Key System Calls

System Call	Description
fork()	Create child process (copy of parent). Returns 0 to child, child PID to parent.
vfork()	Like fork() but child borrows parent's address space (no copy). Child MUST exec() or exit() immediately.
exec*()	Replace process image. execl(path,arg0,...), execv(path,argv[]), execvp(file,argv[]).
wait()/waitpid()	Parent waits for child to terminate; reaps zombie. waitpid(-1,&status;,WNOHANG) non-blocking.
exit(status)	Terminate current process; flush stdio buffers; close files; send SIGCHLD to parent.
getpid()/getppid()	Return current process PID / parent's PID.
pipe(fd[2])	Create unidirectional pipe. fd[0]=read, fd[1]=write. Used between parent and child.
dup2(old,new)	Duplicate file descriptor. dup2(fd,STDOUT_FILENO) redirects stdout to fd.
mmap()	Map files or anonymous memory into process address space. Used for shared memory and file I/O.
shmget/shmat	System V shared memory: create (shmget) and attach (shmat) shared memory segment.

12.3 Important Linux Commands for Interviews

Command	Purpose
ps aux	List all running processes with PID, CPU%, memory, command
top / htop	Real-time process monitor sorted by CPU usage
kill -9 PID	Send SIGKILL to force-terminate process (uncatchable). kill PID sends SIGTERM (catchable).
nice -n 10 cmd	Start command with priority 10 (range -20 to 19; lower = higher priority)

Command	Purpose
renice 5 -p PID	Change priority of running process
lsdf -p PID	List open files for a process
strace -p PID	Trace system calls made by a process
netstat -tulnp	Show listening TCP/UDP ports and processes
df -h	Disk filesystem usage (human-readable)
du -sh dir/	Disk usage of directory
grep -rn 'pattern'	Recursive search with line numbers
awk '{print \$1}'	Text processing — print first field of each line

QUICK REVISION BOX

- ✓ Linux permissions: rwxrwxrwx = owner/group/others. SUID=4, SGID=2, Sticky=1.
- ✓ chmod 755 = rwxr-xr-x. umask 022 makes new files 644 (666 & ~022) by default.
- ✓ fork()→exec()→wait() is the classic shell process creation pattern.
- ✓ pipe(fd[2]): fd[0]=read end, fd[1]=write end. dup2 redirects I/O.
- ✓ SIGTERM (15): catchable graceful shutdown. SIGKILL (9): uncatchable force-kill.
- ✓ ps aux for process list; lsdf for open files; strace to debug system calls.

CHAPTER 13: Advanced OS Concepts

13.1 SMP vs NUMA

Architecture	Description	Example
SMP (Symmetric Multiprocessing)	All CPUs share single physical memory bus. Simple programming model. Memory access time is uniform. Scales poorly beyond ~16-32 CPUs — bus becomes bottleneck.	Early multi-core x86 systems
NUMA (Non-Uniform Memory Access)	Each CPU (socket) has its own local memory bank. Accessing local memory is fast (~50ns). Accessing remote memory (another socket's RAM) is slower (~150-200ns). Scales to hundreds of CPUs. OS must be NUMA-aware for performance.	Modern multi-socket servers (AMD EPYC, Intel Xeon)

13.2 Virtualization

A **hypervisor** (Virtual Machine Monitor, VMM) creates and manages virtual machines — complete software emulations of physical computers.

Hypervisor Type	Description
Type 1 (Bare-Metal)	Runs directly on hardware. OS-less host. Maximum performance. VMware ESXi, Microsoft Hyper-V, Xen.
Type 2 (Hosted)	Runs as an application on a host OS. Easier setup. Slightly more overhead. VMware Workstation, VirtualBox.

13.3 Container Technology

Containers are lightweight alternatives to VMs. They share the host OS kernel but isolate processes using Linux kernel features:

Feature	Role in Containers
Namespaces	Isolate kernel resources: PID (process IDs), NET (network stack), IPC (shared memory, semaphores), MNT (mount points), UTS (hostname). Each container has its own view of the system.
cgroups	Control Groups: limit and account for resource usage — CPU time, memory, disk I/O, network bandwidth — per group of processes.
Union Filesystem	Layered filesystem (OverlayFS): base image layers are read-only, containers add writable layer on top. Enables efficient image sharing.

■ INTERVIEW TIP: VM vs Container

VM: each guest has its own kernel + full OS (GBs overhead). Container: shares host kernel, only app + dependencies (MBs). Containers start in milliseconds; VMs in seconds. VMs provide stronger isolation; containers are more efficient. Docker uses cgroups+namespaces; Kubernetes orchestrates many containers.

13.4 Hyper-Threading (SMT)

Simultaneous Multi-Threading (SMT) or Intel's **Hyper-Threading**: each physical CPU core presents **two logical processors** to the OS. The core has duplicate register sets, program counters, and interrupt handling for

two threads, but shares execution units (ALU, FPU, L1/L2 cache). When one logical thread stalls (cache miss, branch misprediction), the core switches to the other thread — improving instruction throughput by ~15-30%. The OS sees 8 physical cores as 16 logical CPUs.

13.5 Real-Time Systems

Type	Characteristics
Hard Real-Time	Missing deadline is catastrophic (system failure). Requires deterministic, bounded response time. No virtual memory (page faults unpredictable). Examples: pacemakers, aircraft fly-by-wire, anti-lock brakes.
Soft Real-Time	Missing occasional deadline is degraded performance, not failure. Examples: video streaming (late frame = glitch), online gaming, multimedia.

Rate-Monotonic (RM): static priority assignment — tasks with shorter periods get higher priority. Optimal among fixed-priority algorithms. **Earliest Deadline First (EDF):** dynamic priority — task with earliest absolute deadline gets highest priority. Achieves full CPU utilisation on uniprocessors. Optimal among preemptive algorithms but complex implementation.

QUICK REVISION BOX

- ✓ SMP: uniform memory access, bus bottleneck. NUMA: local memory fast, remote slow — scales better.
- ✓ Type 1 hypervisor: bare-metal (ESXi, Xen) — max performance. Type 2: hosted (VirtualBox) — convenient.
- ✓ Containers use Namespaces (PID,NET,IPC,MNT,UTS) + cgroups — lightweight isolation, shared kernel.
- ✓ Hyper-Threading: 2 logical CPUs per physical core; shares ALU. ~15-30% throughput gain.
- ✓ Hard real-time: missing deadline = failure (pacemaker). Soft real-time: degraded quality ok (video).
- ✓ EDF: optimal dynamic priority scheduling. Rate-Monotonic: optimal fixed-priority.

CHAPTER 14: Top 50 Most Asked OS Interview Questions

Each answer below is written as you would deliver it in an interview: starting with a concise definition, building intuition with analogies, then covering technical depth and edge cases. A **Quick Recall** summary ends each answer.

Q1. What is an Operating System and what are its main functions?

An Operating System is the system software that manages hardware and software resources and provides services to application programs. Think of it as the government of a computer — it doesn't do the actual work (applications do), but it controls who can use resources and when, enforces rules, and provides services that everyone needs.

The five core functions are: (1) **Process Management** — creating, scheduling, and terminating processes; (2) **Memory Management** — allocating, deallocating, and protecting memory; (3) **File System Management** — providing the abstraction of named files organised in directories; (4) **I/O Management** — providing uniform interfaces to diverse I/O devices; (5) **Security and Protection** — controlling access and isolating processes from each other.

■ **Quick Recall:** OS = hardware abstraction layer + resource manager + security enforcer.

Q2. What is the difference between a process and a thread?

A **process** is a program in execution — it has its own private address space, file descriptors, and system resources. It is the basic unit of resource ownership. A **thread** is the basic unit of CPU execution within a process. All threads of a process share the process's address space, code, data, and open files, but each thread has its own program counter, registers, and stack.

The key difference is isolation vs. sharing. Processes are isolated — a crash in one process doesn't affect others. Threads are cooperative — they share memory directly (no IPC needed) but a crash in one thread can crash the entire process. Creating a thread is ~10-100× faster than creating a process because thread creation only allocates a stack and copies registers, not an entire address space.

■ **Quick Recall:** Process = resource container (own address space). Thread = execution unit within process (shared address space, own stack+PC).

Q3. Explain the different states of a process.

A process transitions through five fundamental states during its lifetime. **New:** the process is being created — the PCB is allocated and resources are being assigned. **Ready:** the process is in memory, has all resources except the CPU, and is waiting for the scheduler to dispatch it. **Running:** the process's instructions are currently executing on the CPU. On a single-core system, only one process can be running at any instant. **Waiting (Blocked):** the process is waiting for an event — an I/O completion, a semaphore signal, or a timer. It cannot use the CPU even if it's free. **Terminated:** the process has finished; its PCB persists until the parent calls wait() to retrieve the exit status (otherwise it becomes a zombie).

Key transitions: New→Ready (admitted), Ready→Running (scheduled/dispatched), Running→Waiting (I/O or event), Running→Ready (preempted), Waiting→Ready (event completed), Running→Terminated (exit called).

■ **Quick Recall:** 5 states: New→Ready→Running■Waiting; Running→Terminated. Zombie = terminated but not yet reaped.

Q4. What is a PCB (Process Control Block) and what does it contain?

The PCB is the kernel data structure that represents a process. Every process has exactly one PCB, and the OS uses it to save/restore process state during context switches — it's the 'identity card' of a process. In Linux, the PCB is the `task_struct`.

A PCB contains: **Process ID (PID)** — unique identifier; **Process State** — current state (ready/running/waiting); **Program Counter** — address of next instruction to execute; **CPU Registers** — all general-purpose registers, SP, flags — saved here during context switch; **CPU Scheduling Info** — priority, scheduling queue pointers; **Memory Management Info** — page table pointers, base/limit registers; **Accounting Info** — CPU time used, start time; **I/O Status** — list of open files and I/O devices.

■ **Quick Recall:** PCB = process's identity card in the kernel. Contains PID, state, PC, registers, memory info, open files.

Q5. What is a context switch? How does it differ from a mode switch?

A **context switch** is the mechanism of saving the CPU state of one process (into its PCB) and loading the CPU state of another process (from its PCB), allowing the CPU to be shared among multiple processes. It is initiated by: a timer interrupt (time quantum expired), a higher-priority process becoming ready, or the current process blocking on I/O.

Steps: (1) interrupt triggers trap to kernel; (2) save registers + PC of current process to its PCB; (3) update process state; (4) scheduler selects next process; (5) load registers + PC from new process's PCB; (6) return to user mode and execute new process. Context switches involve switching the virtual address space (TLB flush on most architectures), making them expensive (~1-100μs).

A **mode switch** (user mode → kernel mode) happens on every system call and interrupt. It is cheaper — it saves fewer registers and does NOT switch the address space (same process, just elevated privilege level).

■ **Quick Recall:** Context switch: save one process's full state, load another's (address space switch). Mode switch: same process, privilege level change only — much cheaper.

Q6. What is the Critical Section Problem? State its three requirements.

When multiple processes share data, the segment of code where they access shared data is called the **critical section**. The Critical Section Problem is the challenge of designing protocols so that processes can safely execute their critical sections without corrupting shared data.

Three requirements for a valid solution: (1) **Mutual Exclusion** — if process P_i is in its CS, no other process can be in its CS simultaneously; (2) **Progress** — if no process is in its CS and some wish to enter, the selection cannot be postponed indefinitely. Processes in the remainder section cannot block others from entering; (3) **Bounded Waiting** — there is a bound on how many times other processes can enter the CS after a process has requested to enter (preventing starvation).

■ **Quick Recall:** CS Problem requires: Mutual Exclusion (no two at once), Progress (eventual selection), Bounded Waiting (finite wait).

Q7. Explain Peterson's Solution. Does it work on modern CPUs?

Peterson's solution is a software algorithm for two processes to solve the critical section problem using two shared variables: `turn` (whose turn to enter) and `flag[2]` (whether each process wants to enter). Process P_i sets `flag[i]=true` and sets `turn=j` (yields to the other). It then busy-waits while `flag[j]=true AND turn==j`.

It satisfies all three CS requirements: Mutual Exclusion (both can't be in CS simultaneously — `turn` can be only 0 or 1, not both), Progress (if P_j doesn't want CS, P_i enters immediately), Bounded Waiting (P_i waits at most one entry of P_j).

However, Peterson's solution has limitations: (1) it only works for exactly 2 processes; (2) it uses busy-waiting, wasting CPU; (3) on modern out-of-order CPUs, loads and stores can be reordered — Peterson's requires

sequentially consistent memory, which modern CPUs do NOT guarantee without explicit memory barriers (fence instructions).

■ **Quick Recall:** Peterson's: turn + flag[2] variables. Works for 2 processes, but breaks on modern CPUs without memory barriers (store reordering).

Q8. What is a semaphore? Difference between binary and counting semaphores.

A semaphore is an integer variable that is accessed through two atomic operations: **wait(S)** (also called P or down) which decrements S and blocks if $S < 0$, and **signal(S)** (also called V or up) which increments S and wakes a blocked process if any.

A **binary semaphore** (mutex semaphore) has a value of only 0 or 1. It is used to implement mutual exclusion — one process locks (wait), enters the critical section, then unlocks (signal).

A **counting semaphore** ranges over an unrestricted domain. It is used to control access to a pool of N identical resources (e.g., 3 printers, N database connections). Initialised to N, each wait() acquires one resource, each signal() releases one. A counting semaphore with value 0 blocks all requestors; with value 5, it allows 5 simultaneous accesses.

■ **Quick Recall:** Semaphore: integer + wait (decrement+block) + signal (increment+wake). Binary: mutex (0 or 1). Counting: N identical resources.

Q9. What is Deadlock? State the four necessary conditions.

A deadlock is a situation where a set of processes are permanently blocked, each waiting for a resource held by another process in the set. No process can ever make progress. It's like a traffic gridlock at a 4-way intersection where all cars are blocking each other.

Deadlock can occur only if ALL four Coffman conditions hold simultaneously: (1) **Mutual Exclusion** — at least one resource is used in non-sharable mode (only one process at a time); (2) **Hold and Wait** — a process holds at least one resource while waiting for additional resources; (3) **No Preemption** — resources cannot be forcibly taken; a process must voluntarily release them; (4) **Circular Wait** — a closed chain of processes exists where each process holds at least one resource needed by the next process in the chain.

■ **Quick Recall:** Deadlock: circular wait with no progress. 4 conditions (ALL must hold): Mutual Exclusion, Hold-and-Wait, No Preemption, Circular Wait.

Q10. Explain the Banker's Algorithm.

The Banker's Algorithm is a deadlock avoidance algorithm that checks whether a resource request will leave the system in a safe state before granting it. It's named after a banker who never lets their total loan commitments exceed their capital.

Data structures: **Available[m]** (free resources), **Max[n][m]** (max demand per process), **Allocation[n][m]** (currently held), **Need[n][m] = Max - Allocation**.

When process P_i requests resources: (1) If Request > Need: error (exceeded declared max); (2) If Request > Available: block P_i (resources not available); (3) Tentatively grant request (update Available, Allocation, Need); (4) Run Safety Algorithm: find a safe sequence (an order where every process can finish given current + released resources); (5) If safe state: grant permanently; if unsafe: restore old state, block request.

Limitation: requires advance declaration of maximum needs, fixed number of processes, no long-term reservations.

■ **Quick Recall:** Banker's = check if granting request leads to safe state. Safe = safe sequence exists (all processes can eventually finish). Need = Max - Allocation.

Q11. What is virtual memory? How does demand paging work?

Virtual memory is the technique allowing a process to execute even when it's not entirely in physical memory. The process's logical address space can be much larger than physical RAM. Pages are kept on disk and brought into memory only when needed.

Demand paging loads pages only on first access. Each page table entry has a valid/invalid bit. If a process accesses an invalid page, the CPU generates a page fault trap: (1) OS verifies the reference is legal (in the process's allocated space); (2) OS finds a free frame (or evicts a victim via page replacement if none free); (3) OS schedules disk I/O to load the page into the frame; (4) I/O completes; page table updated (frame number, bit=valid); (5) process restarts the faulting instruction — this time it succeeds.

Benefit: programs start quickly (don't need to load entire binary), more processes can run simultaneously (each only loads what it uses), and programs can be larger than RAM.

■ **Quick Recall:** Virtual memory: process sees large address space; only actively-used pages in RAM. Demand paging: load on first access. Page fault → find free frame → disk read → restart instruction.

Q12. Explain FIFO, LRU, and Optimal page replacement algorithms.

Page replacement algorithms choose which memory page to evict when a page fault occurs and no free frame is available.

FIFO: evict the page that has been in memory the longest (oldest). Simple to implement (just a queue). Suffers from Bélády's Anomaly — adding more frames can paradoxically increase page faults.

LRU (Least Recently Used): evict the page that was accessed least recently. Based on the principle of temporal locality — recently used pages are likely to be used again. Has the stack property (no Bélády's Anomaly) but requires hardware support (timestamp counter or software stack). In practice, approximated by the clock algorithm using a reference bit.

Optimal (OPT): evict the page that will not be used for the longest time in the future. Gives the minimum possible page faults. Not implementable in practice (requires future knowledge) but used as a theoretical benchmark.

■ **Quick Recall:** FIFO: evict oldest (Bélády's possible). LRU: evict least recently used (best practical). OPT: evict farthest future use (optimal benchmark).

Q13. What is thrashing and how can it be prevented?

Thrashing occurs when a process doesn't have enough frames to hold its working set — the set of pages it actively uses. The process generates page faults at an extremely high rate, spending more time waiting for pages to be loaded from disk than executing instructions. CPU utilisation collapses because processes are always blocked on I/O.

At the system level, as degree of multiprogramming increases too much, more processes compete for the same frames, each gets fewer frames, all thrash, and CPU utilisation paradoxically decreases despite more processes being active.

Prevention: (1) **Working Set Model** — OS tracks each process's working set (pages accessed in last Δ time units) and ensures enough frames are allocated to hold it; only admit a new process if sum of working sets $\leq$ available frames; (2) **Page Fault Frequency (PFF)** — monitor each process's page fault rate; if too high, give it more frames; if too low, reclaim frames; (3) **Local replacement** — don't let one thrashing process steal frames from others.

■ **Quick Recall:** Thrashing: too few frames → constant page faults → CPU doing I/O not computation. Fix: Working Set Model — ensure each process has its working set in RAM.

Q14. What is a zombie process? How is it created and removed?

A zombie process is a process that has completed execution (called `exit()`) but still has an entry in the process table because its parent has not yet called `wait()` to read its exit status. The zombie consumes no CPU or memory resources — only a PID and a process table slot. It's waiting for its parent to acknowledge its death.

Creation: child calls `exit()` → kernel frees resources but keeps PCB → parent hasn't called `wait()` → child is zombie.

Removal: parent calls `wait()` or `waitpid()` → kernel returns exit status → zombie removed from process table. Alternative: parent installs `SIGCHLD` signal handler that calls `waitpid()` automatically, or uses `SIG_IGN` for `SIGCHLD` (on Linux, this tells kernel to automatically reap children).

Danger: if a server spawns thousands of child processes without reaping them, PIDs fill up and no new processes can be created. Prevention: always call `wait()` for every `fork()`.

■ **Quick Recall:** Zombie: process exited but parent hasn't called `wait()`. Wastes PID + PCB entry only. Fix: parent calls `wait()` / installs `SIGCHLD` handler.

Q15. Difference between Orphan and Zombie processes.

A **zombie** process has finished executing but its parent is still alive and hasn't called `wait()` yet. The zombie exists only as a dead process entry waiting to be reaped.

An **orphan** process is a process whose parent has died (terminated) before the child did. The child is still alive and running. Linux re-parents orphans to `init` (PID 1) or the nearest subreaper (an ancestor process that registered as a subreaper via `prctl(PR_SET_CHILD_SUBREAPER)`). `init` regularly calls `wait()`, so orphans are eventually properly reaped and don't accumulate.

Key difference: zombie's parent is alive (and negligent). Orphan's parent is dead (orphan itself is alive). Zombies are problematic; orphans are generally handled safely by the OS.

■ **Quick Recall:** Zombie: child dead, parent alive but not waited. Orphan: parent dead, child still running → re-parented to `init`. Zombies are dangerous; orphans are safe.

Q16. What is the Producer-Consumer problem? Solve it with semaphores.

The Producer-Consumer (Bounded Buffer) problem: a producer creates items and places them in a shared buffer of size `N`; a consumer removes and processes items. Problems: producer must wait when buffer is full; consumer must wait when buffer is empty; only one should access buffer at a time.

Solution uses three semaphores: **mutex=1** (binary — ensures mutual exclusion on buffer), **empty=N** (counting — number of empty slots; producer waits if 0), **full=0** (counting — number of full slots; consumer waits if 0).

Producer: `wait(empty)` → `wait(mutex)` → add item → `signal(mutex)` → `signal(full)`. Consumer: `wait(full)` → `wait(mutex)` → remove item → `signal(mutex)` → `signal(empty)`. Critical ordering: always wait on counting semaphore (empty/full) BEFORE mutex semaphore — reversing causes deadlock when buffer is full and producer holds mutex.

■ **Quick Recall:** Producer-Consumer: `mutex` + `empty(N)` + `full(0)` semaphores. Producer: `wait(empty)→wait(mutex)→add→signal(mutex)→signal(full)`. Order matters.

Q17. Explain the Readers-Writers problem.

The Readers-Writers problem involves a shared database: multiple readers can read simultaneously (no data corruption since they don't write), but a writer needs exclusive access. Allowing concurrent reads maximises throughput; ensuring exclusive writes prevents corruption.

First solution (readers priority): maintains a **read_count** variable protected by a mutex. First reader entering locks out writers (wait on `rw_mutex`). Subsequent readers just increment `read_count`. Last reader (`read_count==0`) signals `rw_mutex` to allow a writer. Problem: if readers arrive continuously, writer can starve indefinitely.

Second solution (writers priority): a waiting writer prevents new readers from starting. Implemented by an additional semaphore that writers acquire first, blocking new readers. Readers already reading can finish, but no new readers admitted. Risk: readers may starve if writers arrive continuously.

■ **Quick Recall:** Readers-Writers: multiple readers OK simultaneously; writers need exclusive access. Reader priority may starve writers; writer priority may starve readers.

Q18. Explain the Dining Philosophers problem and its solutions.

Five philosophers sit at a round table with one fork between each adjacent pair. To eat, a philosopher needs both the left and right fork. If all simultaneously grab their left fork and wait for their right fork, deadlock occurs (circular wait condition).

Solutions: (1) **Asymmetric**: most philosophers pick left then right; one philosopher picks right then left — breaks circular wait; (2) **Allow only 4 at table**: semaphore table=4 ensures at most 4 philosophers sit simultaneously, guaranteeing at least one can always get both forks; (3) **Both or neither**: use a monitor — a philosopher only picks up forks if both are free (atomic check-and-grab); (4) **Centralised arbitrator**: philosopher asks a waiter (mutex) for permission to pick up forks.

■ **Quick Recall:** Dining Philosophers: deadlock if all pick left fork simultaneously. Solutions: asymmetric pickup, limit occupancy to 4, monitor with atomic both-or-none.

Q19. What is the difference between preemptive and non-preemptive scheduling?

In **non-preemptive** (cooperative) scheduling, once a process gets the CPU, it keeps it until it voluntarily yields — by blocking on I/O, calling a yield system call, or terminating. The OS cannot forcibly take the CPU away. Simple to implement; no race conditions at kernel level; but long jobs block short ones (convoy effect).

In **preemptive** scheduling, the OS can forcibly remove the CPU from a running process — triggered by a timer interrupt (quantum expired) or a higher-priority process becoming ready. Enables fair time-sharing and better response times for interactive users. Requires careful handling of shared kernel data (interrupts can occur at any point, so kernel needs locks).

All modern general-purpose OSes (Linux, Windows, macOS) use preemptive scheduling. Non-preemptive scheduling is used in some embedded/real-time systems where predictability matters more than fairness.

■ **Quick Recall:** Non-preemptive: process keeps CPU until voluntarily yielding. Preemptive: OS can forcibly take CPU (timer interrupt, higher priority). All modern OSes: preemptive.

Q20. Compare FCFS, SJF, and Round Robin scheduling algorithms.

FCFS (First Come First Served): simplest — serve in arrival order. Non-preemptive. Problem: Convoy Effect — short jobs stuck behind a long job. Good only for batch systems where arrival order matters.

SJF (Shortest Job First): provably optimal for minimising average waiting time. Non-preemptive. Problem: cannot know future burst lengths (prediction via exponential averaging); long jobs can starve if short jobs keep arriving.

Round Robin: each process gets a time quantum q . After q , preempted and moved to end of ready queue. Fair for interactive systems. If q is too large → degenerates to FCFS; too small → too much context switch overhead. Average turnaround time is often worse than SJF, but response time is much better. Used in all time-sharing OSes.

Summary: FCFS = simple but inefficient. SJF = optimal average WT but impractical. RR = fair and good for interactive, but higher average TAT.

■ **Quick Recall:** FCFS: simple, convoy effect. SJF: optimal avg WT, needs future knowledge. RR: fair, best response time, quantum size matters critically.

Q21. What is starvation and how is it prevented using aging?

Starvation is a situation where a process waits indefinitely because other processes with higher priority keep arriving and getting served first. The starving process is always ready but never executed.

Starvation occurs in priority scheduling (low-priority processes never run), SJF (long jobs wait forever for short ones to stop arriving), and SSTF disk scheduling (requests near the current head position keep getting served).

Aging is the standard solution: gradually increase the priority of processes that have been waiting for a long time. For example, every 15 minutes in the ready queue, increase a process's priority by 1. Eventually, even a low-priority process will get a high-enough priority to run. After it executes, its priority resets to the original value. This guarantees bounded waiting without sacrificing priority scheduling benefits.

■ **Quick Recall:** Starvation: low-priority process never runs due to high-priority arrivals. Aging: gradually boost priority of long-waiting processes → guarantees bounded wait.

Q22. What is Bélády's Anomaly?

Bélády's Anomaly is a phenomenon where increasing the number of page frames (more memory) actually results in more page faults with the FIFO page replacement algorithm. This is counterintuitive — you'd expect more memory to always reduce page faults.

Example with reference string 1,2,3,4,1,2,5,1,2,3,4,5: with 3 frames FIFO produces 9 page faults; with 4 frames it produces 10 page faults.

Bélády's Anomaly occurs because FIFO doesn't respect the stack property. The stack property states that the set of pages in memory with $n+1$ frames is always a superset of the set with n frames for any reference string — algorithms with this property cannot suffer from Bélády's Anomaly. LRU and Optimal both have the stack property and are immune. FIFO does not have the stack property.

■ **Quick Recall:** Bélády's Anomaly: FIFO can have MORE page faults with MORE frames. Impossible with LRU or OPT (they have the stack property).

Q23. What is IPC? Compare shared memory and message passing.

Inter-Process Communication (IPC) refers to mechanisms that allow processes to communicate and synchronise their actions. Processes need IPC to cooperate on shared tasks.

Shared Memory: a region of memory is mapped into the address spaces of communicating processes. Producer writes to shared region; consumer reads from it. **Speed:** after setup, data transfer happens at memory speed — no system call per message. **Disadvantage:** requires explicit synchronisation (semaphores/mutexes) to prevent race conditions; harder to program correctly.

Message Passing: processes communicate through kernel-mediated `send()` and `receive()` calls. The kernel copies data between processes. **Advantage:** simpler — no synchronisation needed for message integrity (kernel handles it); works across networks. **Disadvantage:** every message requires two system calls + two data copies → higher overhead than shared memory.

Rule of thumb: shared memory for high-throughput local communication; message passing for distributed systems or simpler programming model.

■ **Quick Recall:** Shared memory: fast (memory speed), needs manual sync. Message passing: safe (kernel-mediated), slower (system calls + data copies).

Q24. Explain the concept of a monitor.

A monitor is a high-level synchronisation construct — a programming language / compiler-enforced mechanism that bundles shared data with the procedures that operate on it, and guarantees that only one process can execute within the monitor at any time.

Unlike semaphores (where programmers must manually call wait/signal — easy to make mistakes), monitors provide **built-in mutual exclusion**. If a process calls a monitor procedure, it's automatically given exclusive access. If another process tries to call any monitor procedure simultaneously, it's automatically blocked at the entry queue.

Monitors also have **condition variables** (`x.wait()` — suspend inside monitor and release lock; `x.signal()` — resume one suspended process). Java's synchronized methods implement a monitor. Java's `wait()` and `notify()` implement condition variables. Monitors are safer than semaphores because mutual exclusion cannot be accidentally omitted.

■ **Quick Recall:** Monitor = shared data + procedures with built-in mutual exclusion (one at a time). Condition variables: `x.wait()` (suspend), `x.signal()` (wake one). Java synchronized = monitor.

Q25. What is the difference between paging and segmentation?

Both paging and segmentation divide a program's memory into parts, but they differ fundamentally in approach.

Paging divides logical memory into fixed-size blocks called pages (typically 4KB) and physical memory into frames of the same size. The division is mechanical — the programmer does not see pages; they are transparent. Paging eliminates external fragmentation (any free frame can hold any page) but may cause internal fragmentation (last page may not be full). The page table maps page numbers to frame numbers.

Segmentation divides a program into variable-size logical units that match the program's structure: code segment, data segment, stack, heap. The programmer and compiler are aware of segments. Segmentation causes external fragmentation (segments of different sizes create holes) but no internal fragmentation. Supports easy sharing of code segments between processes.

Modern systems use both: the x86 architecture uses segmentation for protection rings, but Linux uses a flat segmentation model and relies primarily on paging for memory management.

■ **Quick Recall:** Paging: fixed-size, transparent to programmer, no external fragmentation, possible internal fragmentation. Segmentation: variable-size, logical division, possible external fragmentation, easy code sharing.

Q26. What happens during a system call?

A system call is a request from a user-space program to the kernel to perform a privileged operation. Here's the complete sequence: (1) User program calls a library wrapper (e.g., `libc's read()` function). (2) Wrapper stores system call number in a register (e.g., `rax=0` for read on Linux x86-64) and arguments in other registers. (3) Wrapper executes the `SYSCALL` instruction (or `int 0x80` on 32-bit). (4) CPU switches from Ring 3 to Ring 0 (kernel mode), saves user registers on the kernel stack, and jumps to the syscall dispatch table. (5) Kernel's syscall handler looks up the handler function using the syscall number. (6) Handler executes the requested service (read from file, etc.). (7) Handler stores return value in `rax`, restores user registers, executes `SYSRET` to switch back to Ring 3. (8) Library wrapper returns result to user program.

Critical insight: system calls are expensive — typically 100-300ns — because they involve mode switches, kernel stack switches, and potential TLB and cache pollution. Modern techniques like `io_uring` (Linux) batch system calls to reduce this overhead.

■ **Quick Recall:** System call: user program → trap/syscall instruction → mode switch to kernel → kernel handler → `sysret` → return to user mode. ~100-300ns overhead.

Q27. What is the difference between hard and soft real-time systems?

In a **hard real-time system**, every task has an absolute deadline that must be met; missing a deadline is a catastrophic system failure. These systems have deterministic behavior — the worst-case execution time of every task must be known in advance. They typically avoid virtual memory (page faults are unpredictable), use static priority assignment, and run on dedicated hardware. Examples: pacemakers (late heartbeat = patient dies),

aircraft fly-by-wire, anti-lock braking systems, nuclear reactor control.

In a **soft real-time system**, deadlines are important and the system tries to meet them, but occasional missed deadlines result in degraded performance, not system failure. Examples: video streaming (a late frame causes a glitch), online gaming (late update causes lag), multimedia applications. Soft real-time systems can use normal OS features including virtual memory.

■ **Quick Recall:** Hard real-time: missing deadline = system failure (pacemaker, aircraft). Soft real-time: missing deadline = degraded quality (video stream). Hard RT avoids VM; soft RT can use normal OS.

Q28. What is Copy-on-Write (COW) and how does it optimise fork()?

Copy-on-Write is a resource management optimisation used to defer expensive copy operations until they are actually necessary. In the context of fork(), instead of immediately copying all parent pages to the child, both parent and child initially share the same physical pages marked as read-only.

When either process (parent or child) attempts to write to a shared page, a protection fault is triggered. The OS then creates a private copy of that page for the writing process and marks both copies as writable. Only pages that are actually modified are ever copied.

Benefit: fork() becomes nearly instantaneous — only the page table is copied, not the actual data. If the child immediately calls exec() (as a shell does), the shared pages are never written to, so no copying ever occurs. This makes the fork-exec idiom extremely efficient. COW also benefits memory-mapped files and kernel page caches.

■ **Quick Recall:** COW: after fork(), parent and child share pages (read-only). On write, only then copy that page. Makes fork() $O(\text{page_table_size})$ instead of $O(\text{address_space_size})$.

Q29. Explain Multilevel Feedback Queue (MLFQ) scheduling.

MLFQ is the most sophisticated practical scheduling algorithm, designed to achieve the benefits of SJF without requiring advance knowledge of burst times. It uses multiple queues with different priorities and time quanta, and allows processes to move between queues based on their observed behavior.

Newly arriving processes enter the highest priority queue (shortest quantum). If a process uses its entire quantum without blocking, it's demoted to the next lower queue (longer quantum). Processes that frequently block before their quantum expires (I/O-bound) stay in higher queues. Lower queues are scheduled only when all higher queues are empty.

Aging prevents starvation: after a process has waited in a low-priority queue for too long, it's promoted to a higher queue. This gives it a chance to run and be reclassified based on current behavior.

MLFQ effectively learns a process's nature: CPU-bound processes settle in low-priority queues with long quanta; I/O-bound (interactive) processes stay in high-priority queues with short quanta. This matches the ideal of running interactive processes fast and batch jobs efficiently.

■ **Quick Recall:** MLFQ: multiple queues with decreasing priority and increasing quantum. New processes start high, demote if CPU-bound, promoted (aged) to prevent starvation. Best practical scheduler.

Q30. What is the difference between internal and external fragmentation?

Internal fragmentation is wasted space inside an allocated memory block. It occurs when the allocated block is larger than the actual request. Example: if the allocator works in 32-byte blocks and a request needs 20 bytes, 12 bytes are wasted inside the block. In paging, the last page of a process may not be full — up to $(\text{page_size} - 1)$ bytes are wasted per process.

External fragmentation is wasted space outside all allocated blocks — free memory is fragmented into many small, non-contiguous pieces. Even if total free memory is sufficient for a request, no single contiguous block is large enough. In variable-partition memory allocation, frequent alloc/free operations leave scattered holes. In

paging, external fragmentation doesn't occur because any free frame can hold any page.

Solution for external fragmentation: compaction (expensive — move all allocated blocks together), or use paging/non-contiguous allocation. Solution for internal fragmentation: use smaller block sizes (increases overhead) or best-fit allocation.

■ **Quick Recall:** Internal fragmentation: wasted space INSIDE allocated block (paging). External fragmentation: free space scattered outside blocks, no contiguous chunk large enough (variable partitions).

Q31. What is the role of the TLB in virtual memory?

The Translation Lookaside Buffer is a small, fully associative hardware cache inside the CPU's MMU (Memory Management Unit) that stores recent virtual-to-physical address translations (page→frame mappings). Without TLB, every memory access would require two memory accesses: one to the page table in RAM (to find the frame number) and one for the actual data. This would double effective memory access time.

With TLB: CPU first checks TLB (1-2 CPU cycles). If the translation is there (TLB hit, ~99% of the time with good locality), the physical address is found in 1-2 cycles. If not (TLB miss), the page table must be walked (hardware page table walker on x86, or software TLB miss handler on MIPS), the translation is loaded into TLB, and the access retried.

TLB contains 64-1024 entries. On a context switch, TLB is flushed (or tagged with Address Space IDs — ASIDs — to allow multiple processes' translations to coexist). $EAT = \alpha(TLB+mem) + (1-\alpha)(TLB+2mem)$, where α is TLB hit ratio.

■ **Quick Recall:** TLB: fast cache for virtual→physical translations. ~99% hit ratio. TLB miss → page table walk. Flushed on context switch (or tagged with ASID).

Q32. What is thrashing? How does the Working Set Model prevent it?

Thrashing is a severe performance degradation where processes spend more time paging (waiting for disk I/O) than executing. It occurs when the total demand for memory frames exceeds physical RAM: processes constantly page fault because their actively-needed pages are being evicted to make room for others.

The Working Set Model (Denning, 1968) defines a process's working set $W(t, \Delta)$ as the set of pages it has referenced in the last Δ time units (the 'working set window'). The process needs at least $|W(t, \Delta)|$ frames to avoid thrashing.

Prevention: the OS monitors each process's working set size. It only allows new processes to start if $\text{sum}(\text{working set sizes}) \leq \text{total available frames}$. If the system is overcommitted, the OS suspends (swaps out) some processes entirely to allow remaining processes sufficient frames. This is a key difference from just limiting multiprogramming degree — it's adaptive to what processes are actually using.

■ **Quick Recall:** Thrashing: too few frames → constant page faults → CPU idle (waiting on I/O). Working Set Model: track pages used in last Δ time; ensure each process has its working set in RAM.

Q33. What are the different types of OS kernels? Compare monolithic and microkernel.

A monolithic kernel runs the entire OS in a single large kernel-space binary. All OS services — file system, device drivers, process scheduler, memory manager, network stack — run in kernel space at the highest privilege level. Advantages: excellent performance (function calls instead of IPC between components), no context switching between services. Disadvantage: a bug in any driver crashes the entire system; hard to maintain as the code grows (Linux kernel is >25 million lines).

A microkernel keeps only the bare minimum in the kernel: basic IPC, thread scheduling, and memory management. Everything else (file system, device drivers, network stack) runs as user-space servers. The kernel is very small and verifiable. Advantages: fault isolation (a crashing driver doesn't crash the kernel), easy to port, highly modular. Disadvantage: every service interaction requires multiple IPC calls

(user→kernel→server→kernel→user), making it significantly slower than monolithic.

Hybrid kernels (Windows NT, macOS XNU) move some services into user space while keeping performance-critical components in kernel space — a pragmatic middle ground.

■ **Quick Recall:** Monolithic (Linux): all in kernel space, fast, hard to isolate faults. Microkernel (QNX): minimal kernel, services in user space, slow IPC but fault-tolerant. Hybrid (Windows): best of both.

Q34. Explain disk scheduling. Which algorithm is best and why?

Disk scheduling determines the order in which pending disk I/O requests are serviced. The dominant cost is seek time — moving the disk read/write head to the correct track. Minimising total head movement minimises average service time.

Among common algorithms: FCFS is fair but gives worst total head movement. SSTF minimises each individual seek but causes starvation (requests far from head never served). SCAN (elevator) moves head in one direction, serves all requests, then reverses — good throughput, no starvation. LOOK is like SCAN but only travels to the last request in each direction (doesn't waste time going to disk edge). C-LOOK provides more uniform wait times.

In practice, **C-LOOK** or **LOOK** are the recommended algorithms for general-purpose OSes — they provide good throughput without starvation and more uniform response times than SCAN. Linux's deadline I/O scheduler uses a variant of LOOK with separate deadline queues to prevent starvation of old requests.

■ **Quick Recall:** Disk scheduling minimises head movement. Best general-purpose: C-LOOK (good throughput, uniform wait, no starvation). Linux uses deadline scheduler.

Q35. What are RAID levels? Explain RAID 0, 1, 5, and 10.

RAID (Redundant Array of Independent Disks) combines multiple physical disks to improve performance, reliability, or both.

RAID 0 (Striping): data is striped across disks in blocks. Both disks are read/written in parallel — maximum read/write throughput and full capacity. No redundancy — if any disk fails, all data is lost. Used where performance matters and data loss is acceptable (scratch disks, caches).

RAID 1 (Mirroring): every bit written to one disk is simultaneously written to a mirror disk. If one fails, the other has all data. Full redundancy at 50% capacity cost. Read performance can be doubled (read from either disk).

RAID 5 (Distributed Parity): data and parity blocks striped across 3+ disks. Parity allows reconstruction of any one failed disk. Capacity: (N-1)/N of total. Good balance of performance, capacity, and redundancy. Write performance is reduced (must compute and write parity). Most common in enterprise storage.

RAID 10 (Stripe of Mirrors): RAID 1+0 — first mirror pairs of disks, then stripe across mirrors. Excellent read/write performance + can survive multiple drive failures (as long as not both disks of a mirror pair). 50% capacity overhead.

■ **Quick Recall:** RAID 0: max performance, no redundancy. RAID 1: full mirror, 50% overhead. RAID 5: distributed parity, 1 disk fault tolerance. RAID 10: mirror then stripe, best performance+resilience, 50% overhead.

Q36. What is the Ostrich Algorithm for deadlock handling?

The Ostrich Algorithm is the simplest approach to deadlock: ignore the problem entirely and pretend it doesn't exist — like an ostrich burying its head in the sand. If a deadlock occurs, the user detects the system hang and restarts the affected process or reboots.

Rationale: deadlock prevention requires severe constraints (e.g., requesting all resources at once); deadlock avoidance (Banker's Algorithm) requires knowing future resource requests in advance and adds overhead to every allocation; detection adds continuous monitoring overhead. For general-purpose systems where deadlocks are rare, the cumulative cost of prevention/avoidance/detection exceeds the cost of occasional deadlocks.

Linux, Windows, macOS, and most UNIX systems effectively use the Ostrich Algorithm — they make no attempt to prevent or detect deadlocks in the general case. Some specific subsystems (e.g., database systems) implement deadlock detection within their own framework.

■ **Quick Recall:** Ostrich Algorithm: ignore deadlocks. Used by Linux, Windows, macOS. Rationale: deadlocks are rare; prevention/avoidance overhead exceeds benefit.

Q37. Explain the concept of paging with a Translation Lookaside Buffer.

Paging divides logical memory into fixed-size pages and physical memory into same-size frames, maintaining a page table to map page numbers to frame numbers. The page table can be large (4MB for a 32-bit process with 4KB pages). Without hardware support, every memory access requires two accesses: one to read the page table, one for the actual data.

The TLB solves this by caching recent page-to-frame translations in a small, fast hardware structure inside the CPU. When the CPU generates a virtual address, the MMU first searches the TLB. On a TLB hit (>99% of accesses with good locality), the frame number is returned in 1 cycle. On a TLB miss, the MMU walks the page table (multiple memory accesses), loads the translation into the TLB (evicting an old entry), and then retries.

Effective Access Time = $\alpha \times (t_{tlb} + t_{mem}) + (1-\alpha) \times (t_{tlb} + 2 \times t_{mem})$, where α is hit ratio. With $\alpha=0.99$, $t_{tlb}=10ns$, $t_{mem}=100ns$: $EAT = 0.99 \times 110 + 0.01 \times 210 = 111ns$ (vs 210ns without TLB).

■ **Quick Recall:** TLB: hardware cache for page→frame mappings inside MMU. 99% hit rate. $EAT = \alpha(TLB+MEM) + (1-\alpha)(TLB+2MEM)$. TLB flushed on context switch or tagged with ASID.

Q38. What is the significance of the fork() system call?

fork() is the fundamental Unix mechanism for creating new processes. It creates an exact copy of the calling process: the child gets duplicates of parent's address space (using Copy-on-Write), file descriptors, signal handlers, and current working directory. The child is a new process with a new PID.

Return value: fork() returns 0 in the child and the child's PID in the parent. This allows one code path to distinguish parent from child behavior. A return value of -1 indicates failure (process table full, no memory).

The fork-exec pattern is the Unix way to launch programs: fork() creates a copy of the shell, then exec() replaces the child with the desired program. This design is elegant — programs don't need to know how to start other programs; the shell handles all setup (I/O redirection, pipes) between fork() and exec().

fork() without subsequent exec() is used for parallel computation: parent and child do different parts of a calculation, then communicate via pipe or shared memory. Modern alternatives: posix_spawn() (combines fork+exec atomically), clone() (Linux-specific, more control over what's shared — the basis for threads and containers).

■ **Quick Recall:** fork(): creates exact copy of process. Returns 0 to child, child PID to parent. Fork+exec = how shells launch programs. Uses COW for efficiency.

Q39. What is a named pipe (FIFO)? How does it differ from an unnamed pipe?

An **unnamed pipe** (created with pipe()) is a unidirectional IPC channel that exists only between a parent process and its children (processes that share the pipe file descriptors). It exists only in memory; has no name in the filesystem; and disappears when all processes close their ends. Limitation: can only be used between related processes (parent-child or siblings).

A **named pipe (FIFO)** is a special filesystem entry created with mkfifo() or mknod(). It has a name in the filesystem (e.g., /tmp/myfifo) and any process that knows the path can open it for reading or writing. Unlike files, data written to a FIFO goes to a kernel buffer and is read by the reading process — data is not stored on disk. Named pipes persist in the filesystem until deleted; they enable IPC between completely unrelated processes.

Both are half-duplex (one direction per pipe). For bidirectional IPC between unrelated processes, use two named pipes, message queues, or sockets.

■ **Quick Recall:** Unnamed pipe: parent-child only, memory-only, one direction. Named pipe (FIFO): filesystem entry, any two processes, kernel-buffered, persists until deleted.

Q40. Explain CPU burst and I/O burst. Why does it matter for scheduling?

Processes alternate between two phases during execution. A **CPU burst** is a period where the process is actively executing instructions on the CPU. An **I/O burst** is a period where the process is waiting for an I/O operation to complete (disk read, network packet, keyboard input). During the I/O burst, the process is blocked and cannot use the CPU even if it's available.

This matters enormously for scheduling: **I/O-bound processes** have many short CPU bursts (a few milliseconds each) interspersed with many I/O waits. They need fast response time — if they wait long for the CPU, they make the I/O device sit idle. **CPU-bound processes** have few, very long CPU bursts. They need high throughput but can tolerate delays between bursts.

Good schedulers like MLFQ automatically detect process type from behavior: I/O-bound processes voluntarily release the CPU before their quantum expires, so MLFQ keeps them in high-priority queues with short quanta. CPU-bound processes use their entire quantum and get demoted to lower queues with longer quanta — less context switching overhead.

■ **Quick Recall:** CPU burst: process executing. I/O burst: process waiting for I/O. I/O-bound: many short CPU bursts. CPU-bound: few long CPU bursts. Good schedulers optimise for both.

Q41. What is a Mutex? How does it differ from a Semaphore?

A **mutex** (mutual exclusion lock) is a binary synchronisation primitive. It has two states: locked and unlocked. Only one thread can hold a mutex at any time. The key rule: **only the thread that locked a mutex can unlock it** — ownership semantics. Mutex operations: `lock()` / `acquire()` — blocks until unlocked, then locks; `unlock()` / `release()` — releases the lock.

A **semaphore** is a more general synchronisation primitive — an integer counter with `wait()` and `signal()` operations. Binary semaphores (0/1) behave similarly to mutexes but lack ownership semantics: any thread can signal a semaphore that another thread is waiting on. Counting semaphores manage pools of resources (e.g., N database connections).

Key differences: (1) Ownership — mutex has owner (only locker can unlock); semaphore has no owner (any thread can signal). (2) Initial value — mutex starts unlocked; semaphore starts at N (resource count). (3) Use case — mutex for mutual exclusion only; counting semaphore for resource management. (4) Priority inversion — mutexes can implement priority inheritance (kernel elevates mutex holder's priority); semaphores typically cannot.

■ **Quick Recall:** Mutex: binary lock with ownership (only locker can unlock). Semaphore: integer counter, no ownership, used for mutual exclusion (binary) or resource pools (counting).

Q42. What is priority inversion? How is it solved?

Priority inversion occurs when a high-priority task is indirectly blocked by a low-priority task — violating the fundamental assumption that high-priority tasks should always preempt low-priority ones. Classic scenario: L (low priority) holds a lock. H (high priority) tries to acquire the lock and blocks. M (medium priority) arrives and preempts L (since H is blocked, M can run). Now M prevents L from running, which prevents H from getting the lock. H (highest priority) is indirectly blocked by M (medium priority) indefinitely.

Famous real-world example: Mars Pathfinder (1997) — a priority inversion bug caused the system watchdog to reset the spacecraft computer repeatedly. Fixed in-flight by enabling priority inheritance.

Solutions: (1) **Priority Inheritance:** when L holds a lock that H wants, temporarily elevate L's priority to H's priority so L can finish quickly and release the lock. After releasing, L's priority returns to normal. (2) **Priority Ceiling Protocol:** assign each resource a ceiling priority equal to the highest priority of any task that might lock it. A task can only lock a resource if its priority is higher than the ceiling of all currently locked resources.

■ **Quick Recall:** Priority inversion: H blocks on lock held by L, then M preempts L → H waits on M. Fix: Priority Inheritance (elevate L to H's priority while it holds the lock).

Q43. What is demand paging vs prepaging?

Demand paging loads pages into memory only when they are first accessed — on demand. No page is loaded until a process tries to access it. Advantages: fast process startup (don't load entire program), efficient memory use (only needed pages loaded). Disadvantage: initial access to each new page causes a page fault (costly disk I/O). Most modern OSes use demand paging.

Prepaging (prefetching) loads pages into memory before they are actually requested, anticipating future accesses. The OS predicts which pages will be needed next (based on sequential access patterns or process history) and loads them proactively. Advantages: reduces page fault rate by eliminating some disk access latency. Disadvantage: wastes I/O bandwidth and memory if predictions are wrong (loaded pages that were never accessed).

Practice: modern systems combine both — demand paging as default, with read-ahead (prepaging) for sequential file access (kernel reads the next few blocks ahead when it detects sequential access pattern).

■ **Quick Recall:** Demand paging: load on first access (page fault). Prepaging: load anticipated pages proactively. Modern OSes: demand paging + sequential read-ahead.

Q44. What is the difference between a thread library in user space vs kernel space?

A **user-space thread library** manages threads entirely within the user process, without any kernel involvement for thread creation, context switching, or synchronisation. The library maintains its own thread table and scheduler. The kernel sees only a single-threaded process. Examples: GNU Pth, early POSIX thread implementations. Advantages: very fast thread operations (no system calls); portable (runs on any OS); thread-local scheduling policies. Disadvantages: if one thread makes a blocking system call, the entire process blocks (kernel sees one process, all user threads stop); cannot run in parallel on multi-core CPUs (kernel only schedules one unit).

A **kernel-space thread library** has the kernel directly manage threads. Each thread is a kernel object with its own entry in the kernel's scheduler. The kernel performs thread creation, context switching, and synchronisation via system calls. Examples: Linux NPTL (Native POSIX Thread Library) — 1:1 model. Advantages: true parallelism on multi-core; one blocking thread doesn't block others. Disadvantages: thread operations are system calls (slower than user-space); more kernel overhead.

■ **Quick Recall:** User-space threads: fast, portable, no parallelism, blocking I/O blocks all threads. Kernel threads (NPTL): true parallelism, real blocking, slower creation. Linux uses 1:1 kernel threads.

Q45. What is the Safe State in the context of the Banker's Algorithm?

A system is in a **safe state** if there exists a **safe sequence** — an ordering of all processes P₁, P₂, ..., P_n such that: for each process P_i, the resources it may still request can be satisfied by the currently available resources plus the resources held by all processes P_j where j < i (processes that have already been granted resources and finished in the sequence).

Intuition: a safe state means we can find an order to run all processes to completion, even if they all request their maximum resources simultaneously. Each process finishes and releases its resources, enabling the next in the sequence to proceed.

An **unsafe state** is not necessarily a deadlocked state — it means there's a possibility of deadlock if processes make their maximum requests simultaneously. The Banker's Algorithm avoids entering unsafe states: it only grants resource requests that leave the system in a safe state.

Safe state → no deadlock possible (given the safe sequence). Unsafe state → deadlock may occur (not guaranteed, but possible). The Banker's Algorithm is conservative: it may deny requests that wouldn't actually cause deadlock.

■ **Quick Recall:** Safe state: a safe sequence exists where all processes can eventually finish. Banker's Algorithm: only grant requests that keep system in safe state. Unsafe ≠ deadlock, but deadlock is possible.

Q46. How does Linux implement processes and threads differently?

In Linux, there is actually no fundamental distinction between processes and threads at the kernel level. Both are represented by the same kernel data structure: `task_struct`. The difference is in what resources they share.

In Linux, `fork()` creates a new `task_struct` that copies parent's resources. `clone()` is the underlying system call that creates both processes (with thread libraries) and threads, with fine-grained control over what's shared: `CLONE_VM` (share virtual memory), `CLONE_FILES` (share file descriptors), `CLONE_SIGHAND` (share signal handlers), `CLONE_THREAD` (share thread group). When you create a pthread on Linux (via `pthread_create()`), it internally calls `clone()` with `CLONE_VM|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD` — creating a task that shares the address space.

All tasks sharing the same `mm_struct` (memory descriptor) and `task_struct.tgid` (thread group ID) are considered threads of the same process. `getpid()` returns `tgid` (same for all threads in a process); `gettid()` returns the individual task's PID.

■ **Quick Recall:** Linux: no kernel distinction between process and thread — both are `task_struct`. `fork()` copies resources; `clone()` shares them selectively. pthreads use `clone()` with `CLONE_VM|CLONE_FILES`.

Q47. What is memory-mapped I/O and DMA?

Memory-Mapped I/O (MMIO): device registers are mapped into the CPU's physical address space. The CPU reads/writes device registers using normal load/store instructions to specific addresses. No special I/O instructions needed. The CPU and device controller share the bus, and the memory controller routes accesses to the appropriate device. Advantage: simple programming model; disadvantage: device accesses go through the CPU and tie up the bus.

DMA (Direct Memory Access): a DMA controller allows devices to transfer data directly to/from system RAM without CPU involvement. The CPU programs the DMA controller (source, destination, length), then returns to other work. The DMA controller performs the transfer autonomously using bus mastering. When done, it interrupts the CPU. This is essential for high-bandwidth devices (disks, NICs, GPUs) because transferring large amounts of data through CPU registers would waste enormous CPU cycles.

Cycle Stealing: on older systems, DMA controllers 'steal' bus cycles from the CPU. The CPU detects when the DMA wants the bus and pauses for one cycle. Modern systems use separate memory buses, so CPU and DMA can often proceed in parallel.

■ **Quick Recall:** MMIO: device registers in CPU address space, accessed with normal load/store. DMA: device transfers data directly to RAM; CPU only involved at start/end interrupt.

Q48. What is the difference between logical and physical addresses?

A **logical address** (also called virtual address) is generated by the CPU during program execution. It is what the running program believes is its address — the program always works with logical addresses, never aware of where it is physically in RAM. All address arithmetic in a program uses logical addresses.

A **physical address** is the actual address used to access physical RAM hardware. The memory chips respond to physical addresses, not logical ones.

The Memory Management Unit (MMU) translates logical to physical addresses at hardware speed, transparently to the program. This separation enables: (1) **Process isolation** — two processes using the same logical address 0x1000 map to different physical addresses; (2) **Relocation** — the OS can move a process to a different physical location without changing the program; (3) **Virtual memory** — logical address space can be larger than physical RAM; (4) **Sharing** — two processes can map different logical addresses to the same physical address (shared libraries).

■ **Quick Recall:** Logical (virtual) address: what the CPU generates; what the program sees. Physical address: actual RAM location. MMU translates at hardware speed; enables isolation, relocation, and virtual memory.

Q49. What is spooling? Give an example.

SPOOL stands for Simultaneous Peripheral Operations On-Line. Spooling is a process where data is temporarily stored (spooled) on disk and then fed to a slower device at its own pace, allowing the fast CPU to continue other work.

Classic example: a **printer spooler**. When you print a document, the OS doesn't make your application wait for the slow printer. Instead, it copies the print job to a disk spool file and returns control to your application immediately. A background print daemon (spooler) reads from the spool file and sends data to the printer at the printer's speed. Multiple users can submit print jobs simultaneously; the spooler queues them and sends one at a time. This is why you can continue working immediately after clicking Print.

Other examples: email delivery (emails are queued in a spool on the mail server, sent when network/recipient server is available), batch job queuing systems. Spooling effectively makes a non-sharable device (printer) appear to be shared among multiple users.

■ **Quick Recall:** Spooling: buffer device output to disk, let slow device serve from spool while CPU continues. Classic example: print spooler — multiple users submit jobs, printer processes them sequentially.

Q50. What are cgroups and namespaces in Linux? How do they relate to containers?

Namespaces provide isolation of kernel resources: each namespace gives a process its own view of a system resource, isolated from other namespaces. Types: PID namespace (isolated process ID space — PID 1 inside container is not PID 1 on host), NET namespace (isolated network stack with its own interfaces, routing tables, iptables rules), MNT namespace (isolated filesystem mount points — container has its own root filesystem), IPC namespace (isolated shared memory and semaphores), UTS namespace (isolated hostname), USER namespace (isolated user IDs — root inside container isn't root on host).

cgroups (Control Groups) limit and account for resource usage per group of processes. A cgroup can impose limits on: CPU time (how many CPU cycles a group can use per second), memory (maximum RSS, swap), disk I/O (read/write bandwidth and IOPS), network bandwidth. Cgroups also enable accounting (measure how much CPU/memory a group has used) and process suspension.

Containers = namespaces (isolation) + cgroups (resource limits) + union filesystem (layered image). Docker creates a container by calling `clone()` with all namespace flags, placing the new process in a new cgroup with resource limits, and mounting the container image as the root filesystem via OverlayFS.

■ **Quick Recall:** Namespaces: isolate PID/NET/MNT/IPC/UTS/USER — process sees its own private view. cgroups: limit CPU/memory/I/O per process group. Container = namespaces + cgroups + OverlayFS.

CHAPTER 15: 25 Must-Practice Problems

Work through every problem below before your interview. LeetCode concurrency problems are especially high-yield for FAANG-style interviews. GFG problems solidify OS theory.

#	Source	ID	Problem	Topic	Key Concept	URL
1	LeetCode	1114	Print in Order	Thread Synchronization	Semaphore/mutex to enforce ordering	leetcode.com/problems/print-in-order/
2	LeetCode	1115	Print FooBar Alternately	Thread Synchronization	Two semaphores for alternating threads	leetcode.com/problems/print-foobar-alterna...
3	LeetCode	1116	Print Zero Even Odd	Semaphore Counting	3-semaphore coordination	leetcode.com/problems/print-zero-even-odd/
4	LeetCode	1117	Building H2O	Semaphore Resource Pool	Barrier synchronization	leetcode.com/problems/building-h2o/
5	LeetCode	1188	Design Bounded Blocking Queue	Producer-Consumer	Mutex + 2 counting semaphores	leetcode.com/problems/design-bounded-block...
6	LeetCode	1195	Fizz Buzz Multithreaded	Thread Coordination	4 threads, semaphore control	leetcode.com/problems/fizz-buzz-multithrea...
7	LeetCode	1226	Dining Philosophers	Deadlock Avoidance	5-fork semaphore with deadlock prevention	leetcode.com/problems/the-dining-philosoph...
8	LeetCode	146	LRU Cache	Page Replacement / LRU	HashMap + doubly linked list	leetcode.com/problems/lru-cache/
9	LeetCode	460	LFU Cache	Page Replacement / LFU	Min-frequency map + ordered set	leetcode.com/problems/lfu-cache/
10	GFG	N/A	FCFS CPU Scheduling	CPU Scheduling	Compute WT, TAT for FCFS	geeksforgeeks.org/program-for-fcfs-cpu-sch...
11	GFG	N/A	SJF CPU Scheduling	CPU Scheduling	Non-preemptive SJF implementation	geeksforgeeks.org/program-for-shortest-job...
12	GFG	N/A	SRTF CPU Scheduling	CPU Scheduling	Preemptive SJF simulation	geeksforgeeks.org/preemptive-priority-cpu-...
13	GFG	N/A	Round Robin CPU Scheduling	CPU Scheduling	Simulate RR with time quantum	geeksforgeeks.org/program-for-round-robin-...
14	GFG	N/A	Priority Scheduling	CPU Scheduling	Preemptive + aging technique	geeksforgeeks.org/cpu-scheduling-in-operat...
15	GFG	N/A	FIFO Page Replacement	Virtual Memory	Simulate FIFO, count faults	geeksforgeeks.org/page-replacement-algorit...
16	GFG	N/A	LRU Page Replacement	Virtual Memory	Stack-based LRU simulation	geeksforgeeks.org/program-for-least-recent...
17	GFG	N/A	Optimal Page Replacement	Virtual Memory	Future-lookahead simulation	geeksforgeeks.org/optimal-page-replacement..

#	Source	ID	Problem	Topic	Key Concept	URL
18	GFG	N/A	Belady's Anomaly	Virtual Memory	Demonstrate FIFO anomaly with 3 vs 4 frames	geeksforgeeks.org/belady-s-anomaly-in-page-...
19	GFG	N/A	Banker's Algorithm	Deadlock Avoidance	Safety algorithm + resource request	geeksforgeeks.org/bankers-algorithm-in-ope...
20	GFG	N/A	FCFS Disk Scheduling	Disk I/O	Compute total head movement	geeksforgeeks.org/fcfs-disk-scheduling-alg...
21	GFG	N/A	SSTF Disk Scheduling	Disk I/O	Nearest-seek-time simulation	geeksforgeeks.org/sstf-disk-scheduling-alg...
22	GFG	N/A	SCAN Disk Scheduling	Disk I/O	Elevator algorithm simulation	geeksforgeeks.org/scan-elevator-disk-sched...
23	GFG	N/A	C-SCAN Disk Scheduling	Disk I/O	Circular scan simulation	geeksforgeeks.org/c-scan-disk-scheduling-a...
24	GFG	N/A	Producer-Consumer Semaphore	Process Synchronization	Implement with 3 semaphores	geeksforgeeks.org/prod-ucer-consumer-soluti...
25	GFG	N/A	Readers-Writers Problem	Process Synchronization	First and second solutions	geeksforgeeks.org/readers-writers-problem-...

Study Strategy

- LeetCode concurrency (1114-1226): solve all 7 — they directly test OS knowledge of semaphores, mutexes, monitors.
- LRU Cache (146): mandatory — appears in every company's interview. Know HashMap + DLL implementation.
- Banker's Algorithm (GFG #19): implement from scratch — common in written rounds at product companies.
- Work FCFS→SJF→SRTF→RR scheduling problems in order — each builds on the previous.
- For disk scheduling: implement all 6 algorithms and compare total head movement — frequently asked numerically.
- Page replacement: implement FIFO and LRU; know how to demonstrate Bélády's Anomaly for FIFO.
- Time yourself: scheduling/page replacement problems should take under 10 minutes in interviews.

QUICK REVISION BOX

- ✓ LeetCode 1114-1226: 7 concurrency problems covering semaphore, mutex, barrier, and deadlock patterns.
- ✓ LRU Cache (LC 146) + LFU Cache (LC 460): must-know data structure implementations.
- ✓ GFG: complete all 6 scheduling algorithms with hand-traced examples and average WT/TAT calculations.
- ✓ GFG: Banker's Algorithm implementation — safety algorithm + resource-request algorithm.
- ✓ GFG: all 3 page replacement algorithms + Bélády's Anomaly demonstration.
- ✓ GFG: all 4 disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN) with total head movement.